STUDI KASUS NYATA

MODUL PRAKTIKUM

Belajar Laravel 13 dari Nol hingga CRUD Lengkap

Membangun **Sistem Informasi Arsip Dokumen (SIARSIP)** langkah demi langkah — dari instalasi, desain database, hingga fitur upload, unduh, dan keamanan berkas.

Edisi Revisi — Oktober 2026

Disusun berdasarkan Dokumentasi Resmi Laravel — laravel.com/docs

Tingkat: Pemula – Menengah

MODUL

Daftar Isi

Kata Pengantar 5

Panduan Penggunaan Modul 6

BAB 1 — Pendahuluan 7

1.1 Tentang Modul Ini 7

1.2 Mengapa Menggunakan Framework Laravel? 8

1.3 Apa yang Baru di Laravel 13 9

1.4 Studi Kasus: Sistem Informasi Arsip Dokumen (SIARSIP) 10

1.5 Rancangan Struktur Database 11

1.6 Teknologi yang Digunakan 14

BAB 2 — Instalasi dan Konfigurasi Project 17

2.1 Persiapan Environment 17

2.2 Instalasi Laravel 13 18

2.3 Menjalankan Project 19

2.4 Konfigurasi Koneksi Database 19

2.5 Menyiapkan Tampilan dengan Bootstrap 5 21

2.6 Membuat Symbolic Link untuk Penyimpanan File 21

BAB 3 — Desain Database: Model dan Migration 24

3.1 Konsep Model dan Migration 24

3.2 Membuat Model dan Migration Kategori 25

3.3 Mengatur Struktur Tabel kategoris 26

3.4 Membuat Model dan Migration Dokumen 27

3.5 Menjalankan Migration 29

3.6 Relasi dan Mass Assignment pada Model 30

BAB 4 — Menampilkan Data dari Database 35

4.1 Alur MVC yang Digunakan 35

4.2 Membuat Controller 36

4.3 Mendaftarkan Route 37

4.4 Membuat Layout Utama 38

4.5 Menampilkan Daftar Kategori 40

4.6 Menampilkan Daftar Dokumen Arsip 42

BAB 5 — Insert Data (Tambah Arsip Dokumen) 47

5.1 Tambah Data Kategori 47

5.2 Tambah Data Dokumen Arsip dengan Upload File 49

BAB 6 — Menampilkan Detail Data dan Unduh Berkas 55

6.1 Menampilkan Detail Dokumen 55

6.2 Mengunduh Berkas Dokumen 58

BAB 7 — Edit dan Update Data 62

7.1 Edit dan Update Data Kategori 62

7.2 Edit dan Update Data Dokumen 64

BAB 8 — Delete Data (Hapus Arsip) 70

8.1 Menghapus Data Kategori (Menangani Relasi Restrict) 70

8.2 Menghapus Data Dokumen Beserta Berkas Fisiknya 72

8.3 Menambahkan Konfirmasi Hapus pada View 73

BAB 9 — Penyempurnaan: Validasi, Keamanan, dan Akses 75

9.1 Merapikan Validasi dengan Form Request 75

9.2 Middleware Autentikasi 77

9.3 Catatan Keamanan Upload Berkas 78

9.4 Flash Message 79

BAB 10 — Penutup 81

10.1 Ringkasan 81

10.2 Proyek Mandiri 82

10.3 Pengembangan Lanjutan 82

10.4 Referensi 83

Lampiran A — Glosarium Istilah 84

Lampiran B — Error Umum dan Solusinya 86

Kata Pengantar

Modul ini disusun sebagai panduan belajar framework Laravel 13 secara terstruktur dan praktis. Berbeda dengan tutorial pada umumnya yang menggunakan studi kasus generik seperti data produk atau data siswa, modul ini dibangun di atas satu studi kasus yang utuh dan aplikatif: **Sistem Informasi Arsip Dokumen (SIARSIP)**. Setiap konsep Laravel — mulai dari instalasi, Model, Migration, hingga CRUD lengkap — langsung dipraktikkan untuk menyelesaikan kebutuhan nyata pengelolaan arsip dokumen digital.

Pemilihan studi kasus arsip dokumen bukan tanpa alasan. Dibandingkan CRUD teks biasa, sistem arsip menuntut pembaca untuk berhadapan dengan persoalan yang lebih kaya: relasi antar tabel (dokumen berkategori, dokumen dimiliki pengguna), penomoran otomatis yang unik, pengelolaan berkas hasil unggahan, serta aturan integritas data seperti restrict dan cascade pada foreign key. Dengan menyelesaikan modul ini, pembaca tidak hanya paham sintaks Laravel, tetapi juga pola pikir merancang aplikasi yang rapi dan aman.

Setiap bab disusun dengan pola yang konsisten: **tujuan pembelajaran**, **penjelasan konsep** yang dikaitkan dengan studi kasus, **langkah praktik** disertai potongan kode lengkap, **penjelasan bagian per bagian** dari kode tersebut, serta **rangkuman dan latihan** di akhir bab. Pembaca didorong untuk tidak sekadar menyalin kode, melainkan memahami alasan di balik setiap baris yang dituliskan.

Selamat belajar, dan selamat membangun SIARSIP!

Panduan Penggunaan Modul

Agar hasil belajar maksimal, perhatikan panduan berikut sebelum memulai:

Prasyarat

- Memahami dasar bahasa pemrograman PHP (variabel, fungsi, array, dan konsep OOP dasar seperti class).
- Memahami dasar HTML dan cara kerja form (method GET/POST).
- Memahami konsep dasar database relasional (tabel, primary key, foreign key). Penjelasan secukupnya tetap diberikan di dalam modul.

Cara Mengikuti Modul

- **Ikuti bab secara berurutan.** Setiap bab dibangun di atas hasil bab sebelumnya — kode Bab 5 membutuhkan tabel dari Bab 3 dan route dari Bab 4.
- **Ketik ulang kode, jangan hanya copy-paste.** Mengetik ulang melatih ingatan otot dan memaksa Anda membaca setiap baris.
- **Baca kotak "Catatan Penting" dengan saksama.** Kotak tersebut menandai jebakan umum yang sering membuat pemula stuck berjam-jam.
- **Kerjakan latihan di akhir setiap bab** sebelum lanjut ke bab berikutnya.
- **Jika menemukan error,** buka Lampiran B (Error Umum dan Solusinya) sebelum bertanya.

Konvensi Penulisan dalam Modul

Penanda	Arti
`$ perintah` pada blok hijau	Perintah yang diketik di terminal/CMD.
Blok kode abu-abu	Potongan kode PHP/Blade yang ditulis ke dalam file project.
▸ `nama/file.php`	Menunjukkan file mana yang sedang dibahas atau diubah.
`kode` di dalam paragraf	Nama class, method, variabel, atau potongan kode pendek.

BAB 1

Pendahuluan

🎯 TUJUAN PEMBELAJARAN

Setelah mempelajari bab ini, Anda diharapkan mampu:

- Menjelaskan apa itu framework dan mengapa Laravel dipilih untuk membangun SIARSIP.
- Menyebutkan hal-hal baru yang dibawa Laravel 13.
- Mendeskripsikan alur bisnis Sistem Informasi Arsip Dokumen (SIARSIP).
- Menggambarkan rancangan tabel database beserta relasinya.

1.1 Tentang Modul Ini

Modul ini adalah panduan belajar Laravel 13 secara terstruktur: mulai dari instalasi, konsep dasar Model dan Migration, hingga implementasi fitur Create, Read, Update, dan Delete (CRUD) secara lengkap. Berbeda dengan tutorial pada umumnya yang menggunakan studi kasus generik seperti data produk, modul ini disusun langsung dengan studi kasus yang aplikatif, yaitu **Sistem Informasi Arsip Dokumen (SIARSIP)**, sehingga Anda dapat langsung memahami penerapan konsep Laravel pada kebutuhan dunia nyata.

Setiap bab disusun dengan pola yang konsisten: penjelasan konsep terlebih dahulu, kemudian langkah praktik, potongan kode lengkap, dan penjelasan bagian demi bagian dari kode tersebut. Dengan pola ini, Anda tidak hanya bisa menyalin kode, tetapi juga memahami alasan dan logika di balik setiap baris yang dituliskan.

Pada akhir modul, Anda akan memiliki sebuah aplikasi arsip dokumen yang berfungsi penuh: dapat menambah kategori, mengarsipkan dokumen beserta berkasnya, mencari dan memfilter arsip, mengunduh berkas, mengubah data, dan menghapus data dengan aman.

1.2 Mengapa Menggunakan Framework Laravel?

Sebelum masuk ke praktik, penting untuk memahami mengapa kita menggunakan framework, bukan menulis PHP dari nol. Sebuah **framework** adalah kerangka kerja — kumpulan pustaka kode, aturan penulisan, dan struktur folder baku — yang menyediakan fondasi siap pakai untuk membangun aplikasi web.

Bayangkan Anda diminta membangun sebuah kantor arsip. Tanpa framework, Anda harus membuat sendiri pintu, jendela, sistem listrik, dan saluran airnya sebelum bisa menata lemari arsip. Dengan framework, "gedung"nya sudah berdiri kokoh: Anda tinggal fokus menata "lemari arsip" sesuai kebutuhan, yaitu fitur-fitur SIARSIP itu sendiri.

Secara konkret, Laravel menghemat pekerjaan Anda pada hal-hal berikut yang semuanya dibutuhkan oleh SIARSIP:

- **Routing** — memetakan URL seperti `/dokumen/5/download` ke kode yang tepat, tanpa menulis parser URL manual.
- **Akses database (Eloquent ORM)** — mengambil dan menyimpan data arsip dengan sintaks PHP yang rapi, tanpa menulis query SQL mentah berulang-ulang.
- **Keamanan bawaan** — perlindungan CSRF pada form, hashing password, dan pelolosan output otomatis pada template, sehingga data arsip lebih aman dari serangan umum.
- **Validasi input** — aturan seperti "judul wajib diisi" atau "file maksimal 10 MB" cukup ditulis satu baris, dan pesan error otomatis tersedia di form.
- **Manajemen file** — API terpadu untuk menyimpan, memeriksa, dan mengunduh berkas arsip ke berbagai media penyimpanan.
- **Autentikasi** — sistem login siap pakai melalui starter kit, sehingga setiap dokumen tercatat siapa pengunggahnya.

> 💡 **ANALOGI**
>
> Jika PHP murni adalah sekotak perkakas tukang, maka Laravel adalah sebuah bengkel yang sudah lengkap dengan meja kerja, rak perkakas yang tertata, dan buku panduan kerja. Anda tetap "tukangnya", tetapi tidak perlu membuat meja kerja dari nol setiap kali ada pekerjaan baru.

1.3 Apa yang Baru di Laravel 13

Laravel 13 melanjutkan arah pengembangan Laravel modern yang berfokus pada kesederhanaan struktur folder, performa, dan kenyamanan pengembang (developer experience). Beberapa poin penting yang perlu dipahami sebelum memulai:

- **Struktur project yang lebih ramping.** Berkas-berkas konfigurasi yang dulu tersebar — seperti Kernel HTTP — kini dipangkas dan disatukan ke dalam `bootstrap/app.php`. Hasilnya, struktur folder project terlihat jauh lebih bersih dan mudah dijelajahi pemula.
- **Routing dan middleware terpusat.** Pendaftaran route dan middleware dilakukan secara terpusat melalui `bootstrap/app.php`, bukan lagi melalui berkas `Kernel.php` seperti pada versi-versi Laravel lama. Ini membuat alur "permintaan masuk → middleware → route" lebih mudah dilacak dalam satu tempat.
- **Dukungan PHP modern (minimal PHP 8.2).** Laravel 13 memanfaatkan fitur-fitur terbaru PHP dengan peningkatan performa pada Eloquent ORM dan query builder — penting ketika data arsip kelak berjumlah ribuan baris.
- **Starter kit autentikasi modern.** Laravel Breeze dan Jetstream menyediakan scaffolding login/register yang siap pakai, sehingga fitur "siapa yang mengunggah dokumen ini" bisa diimplementasikan dengan cepat.
- **Artisan CLI yang semakin powerful.** Pembuatan Model beserta Migration, Factory, Seeder, dan Controller dapat dilakukan dalam satu perintah, misalnya `php artisan make:model Dokumen -m`. Sepanjang modul ini kita akan banyak memanfaatkan kemudahan ini.

Untuk kebutuhan modul ini, kita akan banyak memanfaatkan poin terakhir — Artisan CLI untuk mempercepat pembuatan Model, Migration, dan Controller pada studi kasus Sistem Arsip Dokumen — serta struktur routing terpusat yang membuat kode route SIARSIP mudah dibaca.

> 📚 **RUJUKAN RESMI**
>
> Seluruh konsep pada modul ini merujuk pada dokumentasi resmi Laravel 13 di laravel.com/docs. Jika suatu saat Anda ragu tentang sebuah fitur, dokumentasi resmi adalah sumber kebenaran utama — biasakan membukanya berdampingan dengan modul ini.

1.4 Studi Kasus: Sistem Informasi Arsip Dokumen (SIARSIP)

Skenario Nyata

Bayangkan sebuah kantor dinas yang setiap hari menerima puluhan surat: surat masuk dari instansi lain, surat keluar yang dikirim, sertifikat, kontrak kerja sama, hingga laporan kegiatan. Selama ini, dokumen-dokumen tersebut disimpan sebagai tumpukan kertas di lemari arsip. Akibatnya:

- Mencari sebuah surat lama membutuhkan waktu berjam-jam membongkar map satu per satu.
- Tidak ada catatan siapa yang terakhir memegang atau meminjam dokumen tertentu.
- Dokumen penting berisiko rusak, hilang, atau duplikat tanpa terkendali.

SIARSIP hadir untuk menyelesaikan masalah tersebut: sebuah aplikasi web untuk mencatat dan menyimpan dokumen-dokumen penting suatu instansi secara digital, lengkap dengan kategori dokumen, nomor arsip yang unik, serta berkas fisik yang bisa diunduh kembali kapan saja.

Siapa Penggunanya dan Apa yang Mereka Lakukan?

Peran	Kegiatan di SIARSIP
Petugas Arsip	Membuat kategori arsip (misalnya “Surat Masuk”), mengunggah dan mencatat dokumen baru beserta berkasnya, mengubah data yang keliru, dan menghapus arsip yang sudah tidak berlaku.
Pimpinan / Staf	Mencari dokumen berdasarkan judul atau nomor arsip, memfilter berdasarkan kategori, melihat detail, dan mengunduh berkas untuk keperluan kerja.
Administrator	Mengelola akun pengguna dan memastikan setiap dokumen tercatat siapa pengunggahnya melalui kolom `user_id`.

Dua Entitas Utama

Secara garis besar, sistem ini memiliki dua entitas utama yang akan menjadi tulang punggung seluruh modul:

- **Kategori** — digunakan untuk mengelompokkan dokumen, misalnya "Surat Masuk", "Surat Keluar", "Sertifikat", "Kontrak Kerja Sama", dan sebagainya. Tanpa kategori, ratusan dokumen akan tercampur menjadi satu daftar yang sulit dijelajahi.
- **Dokumen** — merupakan data arsip itu sendiri, berisi judul dokumen, nomor arsip yang unik dan dibuat otomatis, deskripsi, berkas file yang diunggah, tanggal arsip, kategori terkait, dan pengguna yang mengunggah dokumen tersebut.

Sepanjang modul ini, seluruh contoh Model, Migration, Controller, Route, dan View akan langsung mengacu pada dua entitas tersebut, sehingga pada akhir modul Anda telah memiliki sebuah modul CRUD arsip dokumen yang siap dikembangkan lebih lanjut.

1.5 Rancangan Struktur Database

Sebelum menulis satu baris kode pun, perancang aplikasi yang baik selalu memulai dari desain database. Berikut adalah rancangan struktur tabel yang akan digunakan sepanjang modul ini. Struktur ini terdiri atas dua tabel utama — `kategoris` dan `dokumens` — dengan relasi one-to-many antar keduanya, serta relasi ke tabel bawaan Laravel yaitu `users`.

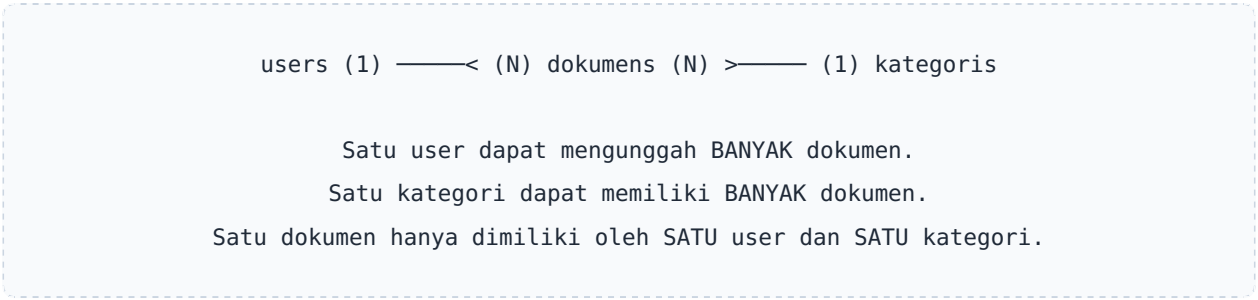


Diagram di atas dibaca sebagai berikut: garis dengan simbol "banyak" (N) menunjukkan sisi yang dapat memiliki banyak baris data. Inilah yang disebut relasi one-to-many: satu kategori (misalnya "Surat Masuk") dapat menaungi ratusan dokumen, tetapi setiap dokumen hanya boleh berada di satu kategori.

a. Tabel kategoris

▸ database/migrations/xxxx_xx_xx_create_kategoris_table.php

```
Schema::create('kategoris', function (Blueprint $table) {
    $table->id();
    $table->string('nama_kategori', 100);
    $table->text('deskripsi')->nullable();
    $table->timestamps();
});
```

Kolom	Tipe Data	Keterangan
`id`	BIGINT UNSIGNED (PK, Auto Increment)	Primary key tabel kategoris — identitas unik setiap kategori.
`nama_kategori`	VARCHAR(100)	Nama kategori dokumen, misalnya “Surat Masuk”. Dibatasi 100 karakter agar tetap ringkas.
`deskripsi`	TEXT (nullable)	Penjelasan tambahan mengenai kategori, boleh kosong (`nullable`).
`created_at` / `updated_at`	TIMESTAMP	Dibuat otomatis oleh Laravel melalui `timestamps()` — mencatat kapan kategori dibuat dan terakhir diubah.

b. Tabel dokumens

▸ database/migrations/xxxx_xx_xx_create_dokumens_table.php

```
Schema::create('dokumens', function (Blueprint $table) {
    $table->id();
    $table->foreignId('kategori_id')->constrained('kategoris')->onDelete('restrict');
    $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
    $table->string('nomor_arsip', 50)->unique();
    $table->string('judul_dokumen', 255);
    $table->text('deskripsi')->nullable();
    $table->string('nama_file', 255);
    $table->string('tipe_file', 20);
    $table->integer('ukuran_file');
    $table->date('tanggal_arsip');
    $table->timestamps();
});
```

Kolom	Tipe Data	Keterangan
`id`	BIGINT UNSIGNED (PK, Auto Increment)	Primary key tabel dokumens.
`kategori_id`	BIGINT UNSIGNED (FK → kategoris.id)	Relasi ke kategori dokumen. `onDelete('restrict')` artinya kategori **tidak bisa dihapus** selama masih memiliki dokumen terkait — melindungi histori arsip.
`user_id`	BIGINT UNSIGNED (FK → users.id)	Relasi ke pengguna yang mengunggah dokumen. `onDelete('cascade')` artinya jika akun user dihapus, seluruh dokumen miliknya ikut terhapus otomatis.
`nomor_arsip`	VARCHAR(50), UNIQUE	Nomor arsip unik yang dibuat otomatis oleh sistem, contoh: `ARS-20260831-0001`. Tidak boleh ada dua dokumen bernomor sama.
`judul_dokumen`	VARCHAR(255)	Judul atau perihal dokumen, misalnya “Surat Undangan Rapat Koordinasi”.
`deskripsi`	TEXT (nullable)	Catatan atau keterangan tambahan dokumen.
`nama_file`	VARCHAR(255)	Nama file hasil unggahan yang tersimpan di server. Disimpan sebagai nama unik (UUID), bukan nama asli, agar tidak saling menimpa.
`tipe_file`	VARCHAR(20)	

		Ekstensi berkas, misalnya `pdf`, `docx`, `jpg` — dicatat otomatis dari file yang diunggah.
`ukuran_file`	INTEGER	Ukuran berkas dalam satuan byte — dicatat otomatis dari file yang diunggah.
`tanggal_arsip`	DATE	Tanggal resmi arsip dokumen. Perhatikan: ini **bisa berbeda** dari tanggal input — misalnya surat bertanggal 1 Agustus baru diarsipkan pada 5 Agustus.
`created_at` / `updated_at`	TIMESTAMP	Dibuat otomatis oleh Laravel.

> ⚠ **CATATAN PENTING**
>
> Perhatikan penggunaan `onDelete('restrict')` pada `kategori_id` dan `onDelete('cascade')` pada `user_id`. Kombinasi ini sengaja dipilih agar data histori arsip tidak pernah kehilangan kategorinya secara tidak sengaja, namun tetap otomatis rapi ketika sebuah akun pengguna dihapus dari sistem. Perbedaan perilaku ini akan kita tangani secara eksplisit pada Bab 8 saat membahas proses hapus data.

> 💡 **MENGAPA MEMISAHKAN KATEGORI KE TABEL SENDIRI?**
>
> Pemula sering tergoda menyimpan kategori sebagai teks bebas di tabel dokumen (misalnya kolom `kategori` berisi "Surat Masuk"). Masalahnya: penulisan yang tidak konsisten ("surat masuk" vs "Surat Masuk") akan merusak filter dan laporan. Dengan tabel kategori tersendiri plus foreign key, ejaan selalu konsisten, penggantian nama kategori cukup dilakukan di satu tempat, dan database menolak kategori yang tidak terdaftar. Inilah inti integritas data.

1.6 Teknologi yang Digunakan

Komponen	Teknologi	Peran dalam SIARSIP
Bahasa Pemrograman	PHP ≥ 8.2	Bahasa utama Laravel; versi 8.2 adalah syarat minimal Laravel 13.

Framework	Laravel 13	Framework utama: routing, Eloquent, validasi, dan keamanan.
Database	MySQL / MariaDB	Menyimpan seluruh data arsip; disarankan XAMPP atau Laragon agar mudah dikelola via phpMyAdmin.
Front-end	Bootstrap 5 (CDN)	Mempercantik tampilan tabel, form, dan tombol tanpa proses build asset tambahan.
Dependency Manager	Composer	Mengunduh dan mengelola package PHP milik Laravel beserta dependensinya.
Text Editor	Visual Studio Code	Disarankan karena dukungan ekstensi PHP/ Laravel yang baik; editor lain tetap bisa digunakan.

📝 Rangkuman Bab 1

- Framework seperti Laravel menyediakan fondasi siap pakai (routing, database, keamanan, validasi) sehingga kita fokus pada fitur bisnis SIARSIP.
- Laravel 13 membawa struktur project yang lebih ramping, routing terpusat di `bootstrap/app.php`, dan Artisan CLI yang powerful.
- SIARSIP mendigitalkan arsip dokumen instansi dengan dua entitas utama: **Kategori** dan **Dokumen**, berelasi one-to-many, plus relasi ke **User** pengunggah.
- Keputusan desain penting: nomor arsip unik otomatis, nama file unik (UUID), `onDelete('restrict')` untuk kategori, dan `onDelete('cascade')` untuk user.

✏ LATIHAN BAB 1

1. Jelaskan dengan kata-katamu sendiri: apa perbedaan menulis aplikasi dengan PHP murni vs dengan framework Laravel?
2. Sebuah instansi memiliki jenis dokumen: Nota Dinas, SK, dan Laporan. Gambarkan isi tabel `kategoris` dan dua contoh baris tabel `dokumens` yang berelasi dengannya.
3. Mengapa kolom `tanggal_arsip` dipisahkan dari `created_at`? Berikan satu contoh kasus nyata.

BAB 2

Instalasi dan Konfigurasi Project

🎯 **TUJUAN PEMBELAJARAN**

- Menyiapkan environment pengembangan (PHP, Composer, web server lokal).
- Membuat project Laravel 13 baru bernama `siarsip`.
- Menjalankan project di browser melalui server bawaan Laravel.
- Menghubungkan project ke database MySQL `db_siarsip`.
- Menyiapkan penyimpanan berkas arsip dengan symbolic link.

2.1 Persiapan Environment

Sebelum melakukan instalasi Laravel 13, pastikan perangkat sudah memiliki komponen-komponen berikut. Setiap komponen memiliki peran yang tidak bisa saling menggantikan:

- **PHP versi 8.2 atau lebih baru** — bahasa yang menjalankan seluruh kode Laravel. Tanpa PHP yang memenuhi syarat, Composer akan menolak menginstal Laravel 13.
- **Composer** — dependency manager PHP. Ia bertugas mengunduh Laravel beserta ratusan pustaka pendukungnya, dan memastikan versinya saling cocok.
- **Web server lokal** seperti XAMPP atau Laragon — menyediakan Apache dan MySQL dalam satu paket, sehingga kita punya database untuk menyimpan arsip tanpa konfigurasi rumit.
- **Node.js dan NPM** (opsional) — hanya diperlukan jika ingin melakukan compile asset front-end. Karena modul ini memakai Bootstrap via CDN, komponen ini boleh dilewati.

Untuk memastikan PHP dan Composer sudah terpasang dengan benar, jalankan dua perintah berikut pada terminal/CMD:

```
php -v
composer -v
```

Jika kedua perintah tersebut menampilkan informasi versi tanpa error, maka environment sudah siap digunakan untuk instalasi Laravel 13. Perintah `php -v` sekaligus menjadi cara cepat memeriksa apakah versi PHP memenuhi syarat minimal 8.2.

> 💡 **TIPS**
>
> Pengguna Windows yang memakai XAMPP perlu memastikan perintah `php` dikenali di CMD — biasanya dengan menambahkan folder `C:\xampp\php` ke PATH environment variable. Jika `php -v` menampilkan pesan "not recognized", masalahnya hampir pasti ada di PATH, bukan di instalasi PHP-nya.

2.2 Instalasi Laravel 13

Ada dua cara umum untuk membuat project Laravel baru: menggunakan **Laravel Installer** (`laravel new`) atau langsung melalui **Composer**. Keduanya menghasilkan project yang sama; Installer hanya memberikan wizard interaktif (pilihan starter kit, database, dll). Modul ini menggunakan cara Composer karena lebih universal di seluruh sistem operasi dan tidak memerlukan instalasi global tambahan:

```
composer create-project laravel/laravel siarsip
```

Perintah di atas akan membuat folder baru bernama `siarsip` yang berisi seluruh struktur dasar project Laravel 13, sekaligus mengunduh seluruh dependency yang dibutuhkan. Nama `siarsip` dipilih agar konsisten dengan studi kasus Sistem Informasi Arsip Dokumen yang digunakan pada modul ini, namun nama folder tersebut bisa disesuaikan sesuai kebutuhan.

Proses ini membutuhkan koneksi internet dan dapat memakan waktu beberapa menit pada unduhan pertama. Tunggu hingga muncul pesan sukses sebelum melanjutkan.

2.3 Menjalankan Project

Setelah proses instalasi selesai, masuk ke dalam folder project, kemudian jalankan built-in server milik Laravel menggunakan Artisan — antarmuka baris perintah resmi Laravel:

```
cd siarsip
php artisan serve
```

Jika berhasil, Laravel akan menjalankan server lokal pada alamat `http://127.0.0.1:8000`. Buka alamat tersebut pada browser, dan halaman default Laravel akan tampil sebagai tanda instalasi berhasil.

> 💡 **ANALOGI**
>
> `php artisan serve` seperti menyalakan "mode demo" SIARSIP di komputer sendiri. Server ini hanya untuk pengembangan — cepat dan praktis, tetapi tidak dirancang untuk melayani pengguna internet secara publik. Nantinya, aplikasi production akan dijalankan melalui web server sungguhan seperti Nginx/Apache.

> ⚠ **CATATAN PENTING**
>
> Jangan tutup jendela terminal selama mengembangkan aplikasi, karena server akan berhenti saat terminal ditutup. Jika port 8000 sudah dipakai aplikasi lain, jalankan `php artisan serve --port=8001` untuk memakai port alternatif.

2.4 Konfigurasi Koneksi Database

Secara default, Laravel 13 dikonfigurasi memakai database SQLite — cukup untuk coba-coba, tetapi untuk SIARSIP kita akan memakai **MySQL** agar sesuai dengan kebutuhan instansi pada umumnya dan mudah dikelola melalui phpMyAdmin.

Langkah pertama: buat database bernama `db_siarsip` melalui phpMyAdmin (tab Databases → Create database) atau perintah SQL `CREATE DATABASE db_siarsip;`. Database-nya masih kosong — tabel-tabel akan dibuat nanti melalui Migration pada Bab 3.

Selanjutnya, buka file `.env` pada root project. File ini menyimpan seluruh konfigurasi yang berbeda-beda antar environment (komputer pengembang, server testing, server production), sehingga **tidak boleh** diikutsertakan ke version control. Cari bagian konfigurasi berikut:

▸ .env (sebelum diubah)

```
DB_CONNECTION=sqlite
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=laravel
# DB_USERNAME=root
# DB_PASSWORD=
```

Ubah konfigurasi tersebut menjadi seperti berikut, disesuaikan dengan studi kasus arsip dokumen:

▸ .env (setelah diubah)

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_siarsip
DB_USERNAME=root
DB_PASSWORD=
```

Baris Konfigurasi	Penjelasan
`DB_CONNECTION=mysql`	Mengubah driver database dari SQLite menjadi MySQL. Baris inilah yang menentukan “bahasa” apa yang dipakai Laravel untuk berbicara dengan database.
`DB_HOST=127.0.0.1`	Alamat server database — `127.0.0.1` artinya database berada di komputer sendiri (localhost).
`DB_PORT=3306`	Port standar MySQL. Ubah hanya jika instalasi MySQL Anda memakai port lain.
`DB_DATABASE=db_siarsip`	Nama database yang akan digunakan untuk menyimpan seluruh tabel arsip.
`DB_USERNAME` / `DB_PASSWORD`	Kredensial database. Nilai `root` tanpa password adalah default XAMPP; sesuaikan jika Anda mengatur password MySQL sendiri.

Setelah disimpan, bersihkan cache konfigurasi agar perubahan langsung diterapkan:

```
php artisan config:clear
```

Perintah ini penting karena Laravel dapat menyimpan konfigurasi dalam bentuk cache. Tanpa `config:clear`, perubahan `.env` kadang tidak terbaca dan Anda akan kebingungan mengapa koneksi database masih gagal.

> ⚠ **CATATAN PENTING**
>
> Pastikan service MySQL pada XAMPP/Laragon dalam keadaan **running** (indikator hijau) sebelum menjalankan migration pada Bab 3. Error “Connection refused” atau “SQLSTATE[HY000] [2002]” hampir selalu berarti MySQL-nya belum dinyalakan, bukan salah konfigurasi Laravel.

2.5 Menyiapkan Tampilan dengan Bootstrap 5

Agar modul ini dapat langsung difokuskan pada logika backend tanpa perlu proses build asset yang rumit (compile dengan Vite/NPM), tampilan antarmuka menggunakan **Bootstrap 5 melalui CDN**. Artinya, file CSS Bootstrap diunduh langsung dari internet saat halaman dibuka — tidak ada file yang perlu diinstal ke project.

Bootstrap akan dipasang langsung di dalam layout utama pada Bab 4 melalui satu baris tag `<link>`, sehingga tidak diperlukan instalasi tambahan melalui NPM. Konsekuensinya: saat mengembangkan, komputer perlu terhubung ke internet agar tampilan ter-render dengan gaya Bootstrap; tanpa internet, halaman tetap berfungsi tetapi tampil polos.

2.6 Membuat Symbolic Link untuk Penyimpanan File

Karena Sistem Arsip Dokumen akan menyimpan berkas hasil unggahan (PDF, Word, gambar, dan sebagainya), kita perlu memahami cara Laravel menyimpan file. Secara default, file yang diunggah disimpan di folder `storage/app/public` — folder yang **tidak bisa diakses langsung** oleh browser demi keamanan.

Agar berkas arsip tetap bisa diakses (diunduh/dilihat) melalui browser, Laravel menyediakan perintah untuk membuat symbolic link — semacam "jalan pintas" — dari

`public/storage` menuju `storage/app/public`. Jalankan perintah Artisan berikut **satu kali saja** setelah instalasi:

```
php artisan storage:link
```

Setelah perintah ini berhasil, setiap file yang disimpan ke `storage/app/public/dokumen/namafile.pdf` otomatis bisa diakses melalui URL `http://127.0.0.1:8000/storage/dokumen/namafile.pdf`.

> ⚠ **CATATAN PENTING**
>
> Tanpa menjalankan perintah `storage:link`, berkas dokumen yang berhasil diunggah tidak akan bisa diakses atau diunduh melalui browser, karena secara default folder `storage/app/public` tidak bisa diakses publik. Gejalanya: upload terlihat sukses, tetapi gambar preview rusak dan tombol unduh error 404. Pastikan langkah ini tidak terlewat sebelum melanjutkan ke Bab 5 (Insert Data).

📝 Rangkuman Bab 2

- Environment SIARSIP: PHP ≥ 8.2, Composer, dan web server lokal (XAMPP/Laragon) untuk MySQL.
- Project dibuat dengan `composer create-project laravel/laravel siarsip`, dijalankan dengan `php artisan serve`.
- Koneksi database diatur di file `.env` (database `db_siarsip`), lalu cache dibersihkan dengan `php artisan config:clear`.
- Tampilan memakai Bootstrap 5 via CDN — tanpa proses build.
- `php artisan storage:link` menghubungkan folder storage ke folder public agar berkas arsip bisa diakses browser.

✏ LATIHAN BAB 2

1. Praktikkan seluruh langkah bab ini hingga halaman default Laravel tampil di browser. Screenshot hasilnya.
2. Sengaja matikan service MySQL, lalu jalankan `php artisan migrate`. Catat pesan error yang muncul, lalu nyalakan kembali MySQL dan ulangi — bandingkan hasilnya.
3. Jelaskan dengan kata-katamu sendiri: apa yang terjadi secara teknis ketika perintah `php artisan storage:link` dijalankan?

BAB 3

Desain Database: Model dan Migration

🎯 **TUJUAN PEMBELAJARAN**

- Menjelaskan peran Model (Eloquent ORM) dan Migration dalam Laravel.
- Membuat Model dan Migration untuk entitas Kategori dan Dokumen.
- Mendefinisikan struktur tabel `kategoris` dan `dokumens` termasuk foreign key.
- Menjalankan migration untuk membuat tabel di database.
- Mendefinisikan relasi Eloquent (`hasMany` , `belongsTo`) dan mass assignment (`$fillable`).

3.1 Konsep Model dan Migration

Dua konsep ini adalah fondasi seluruh modul — pahami dengan baik sebelum mengetik kode.

Model: Representasi Tabel dalam Bentuk Class PHP

Model di Laravel adalah sebuah class PHP yang merepresentasikan satu tabel database melalui **Eloquent ORM** (Object-Relational Mapping). Artinya, setiap baris data di tabel `dokumens` dapat diperlakukan sebagai sebuah object PHP dari class `Dokumen`. Operasi CRUD pun dapat dilakukan dengan sintaks PHP yang rapi tanpa harus menuliskan query SQL secara manual:

```
// Tanpa Eloquent: menulis SQL mentah
$result = mysqli_query($conn, "SELECT * FROM dokumens WHERE kategori_id = 3");

// Dengan Eloquent: sintaks PHP yang ekspresif
$dokumens = Dokumen::where('kategori_id', 3)->get();
```

Selain lebih mudah dibaca, Eloquent otomatis melindungi dari SQL injection dan menyediakan fitur relasi antar tabel yang akan kita pakai intensif di SIARSIP.

Migration: Version Control untuk Database

Migration digunakan untuk mendefinisikan dan mengelola struktur tabel database melalui kode PHP. Dokumentasi resmi Laravel menggambarkannya tepat sebagai "version control for your database" — seperti Git, tetapi untuk skema database.

Mengapa ini penting untuk SIARSIP? Bayangkan tim Anda terdiri dari tiga orang pengembang. Tanpa migration, setiap perubahan struktur tabel (misalnya menambah kolom `ukuran_file`) harus dikomunikasikan manual: "eh, tolong tambahkan kolom ini di database lokalmu ya." Dengan migration, cukup kirim file migration-nya, dan setiap anggota tim menjalankan `php artisan migrate` — struktur database mereka otomatis sinkron, konsisten di semua environment.

> 💡 **ANALOGI**
>
> Jika database adalah sebuah gedung arsip, maka **migration** adalah gambar denah/ renovasinya yang terdokumentasi rapi (kapan tembok ditambah, kapan ruangan dirobohkan), sedangkan **Model** adalah petugas arsip yang tahu persis di rak mana setiap dokumen disimpan dan bagaimana mengambilkannya untuk Anda.

Pada studi kasus Sistem Arsip Dokumen, kita akan membuat dua pasang Model dan Migration, yaitu **Kategori** (untuk tabel `kategoris`) dan **Dokumen** (untuk tabel `dokumens`).

3.2 Membuat Model dan Migration Kategori

Jalankan perintah Artisan berikut untuk membuat Model Kategori sekaligus file Migration-nya secara otomatis:

```
php artisan make:model Kategori -m
```

Opsi `-m` (singkatan dari `--migration`) memerintahkan Artisan untuk langsung membuatkan file migration terkait. Perintah di atas akan menghasilkan dua berkas:

- `app/Models/Kategori.php` — berkas Model
- `database/migrations/xxxx_xx_xx_create_kategoris_table.php` — berkas Migration (xxxx adalah timestamp waktu pembuatan)

Laravel secara otomatis menerjemahkan nama Model `Kategori` menjadi nama tabel `kategoris` (bentuk jamak/plural dan huruf kecil), sehingga kita tidak perlu mendefinisikan nama tabel secara manual. Konvensi ini berlaku umum: Model `Dokumen` → tabel `dokumens`, Model `User` → tabel `users`.

> 💡 **TIPS**
>
> Timestamp pada nama file migration bukan sekadar hiasan — Laravel menggunakannya untuk menentukan **urutan eksekusi** migration. File yang dibuat lebih dulu akan dijalankan lebih dulu. Ini krusial untuk tabel yang memiliki foreign key, seperti yang akan kita lihat pada bagian 3.4.

3.3 Mengatur Struktur Tabel kategoris

Buka file migration yang baru dibuat, lalu ubah method `up()` menjadi seperti berikut. Method `up()` berisi instruksi "membangun" tabel, sedangkan method `down()` berisi instruksi kebalikannya (menghapus tabel) untuk keperluan rollback:

▸ database/migrations/xxxx_xx_xx_create_kategoris_table.php

```
public function up(): void
{
    Schema::create('kategoris', function (Blueprint $table) {
        $table->id();
        $table->string('nama_kategori', 100);
        $table->text('deskripsi')->nullable();
        $table->timestamps();
    });
}
```

Baris Kode	Penjelasan
`Schema::create('kategoris', ...)`	Perintah membuat tabel baru bernama `kategoris`. `Schema` adalah facade yang menyediakan dukungan pembuatan tabel yang database-agnostic — kode yang sama bisa berjalan di MySQL, PostgreSQL, maupun SQLite.
`$table->id();`	Membuat kolom `id` sebagai BIGINT UNSIGNED auto-increment sekaligus primary key. Ini adalah konvensi standar Laravel untuk identitas baris data.

`$table->string('nama_kategori', 100);`	Kolom teks pendek (VARCHAR) maksimal 100 karakter untuk nama kategori seperti “Surat Masuk”.
`$table->text('deskripsi')->nullable();`	Kolom teks panjang (TEXT) untuk deskripsi kategori. Modifier `nullable()` berarti kolom boleh dikosongi — cocok karena tidak semua kategori butuh penjelasan tambahan.
`$table->timestamps();`	Membuat dua kolom `created_at` dan `updated_at` yang diisi otomatis oleh Laravel setiap kali data dibuat/diubah — berguna untuk audit “kapan kategori ini dibuat?”.

3.4 Membuat Model dan Migration Dokumen

Selanjutnya, buat Model dan Migration untuk data dokumen arsip — entitas inti SIARSIP:

```
php artisan make:model Dokumen -m
```

Buka file migration `create_dokumens_table.php` yang baru dibuat, kemudian ubah method `up()` menjadi:

▸ database/migrations/xxxx_xx_xx_create_dokumens_table.php

```
public function up(): void
{
    Schema::create('dokumens', function (Blueprint $table) {
        $table->id();
        $table->foreignId('kategori_id')->constrained('kategoris')->onDelete('restrict');
        $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
        $table->string('nomor_arsip', 50)->unique();
        $table->string('judul_dokumen', 255);
        $table->text('deskripsi')->nullable();
        $table->string('nama_file', 255);
        $table->string('tipe_file', 20);
        $table->integer('ukuran_file');
        $table->date('tanggal_arsip');
        $table->timestamps();
    });
}
```

Beberapa hal penting yang perlu diperhatikan dari kode di atas:

- `foreignId('kategori_id')->constrained('kategoris')` secara otomatis membuat kolom `kategori_id` bertipe BIGINT UNSIGNED **sekaligus** mendefinisikan foreign key ke kolom `id` pada tabel `kategoris`. Satu baris, dua pekerjaan — inilah ekspresivitas Laravel.
- `onDelete('restrict')` mencegah sebuah baris kategori dihapus apabila masih terdapat dokumen yang menggunakan kategori tersebut — ini penting agar histori arsip tidak pernah kehilangan kategorinya. Database akan menolak penghapusan tersebut.
- `foreignId('user_id')->constrained('users')` menghubungkan dokumen dengan tabel `users` bawaan Laravel, yaitu siapa pengguna yang mengunggah/mengarsipkan dokumen tersebut. Tabel `users` sudah tersedia dari migration bawaan Laravel.
- `onDelete('cascade')` pada `user_id` membuat seluruh dokumen milik seorang user otomatis ikut terhapus apabila akun user tersebut dihapus dari sistem — menjaga database tetap bersih dari data yatim.
- `nomor_arsip` diberi properti `->unique()` agar tidak ada dua dokumen dengan nomor arsip yang sama persis. Database sendiri yang akan menolak duplikat, bukan hanya validasi di aplikasi.
- Kolom `nama_file`, `tipe_file`, dan `ukuran_file` mencatat metadata berkas yang diunggah — ketiganya akan diisi **otomatis oleh sistem** pada Bab 5, bukan oleh pengguna.

> ⚠ **CATATAN PENTING**
>
> Karena tabel `dokumens` memiliki foreign key ke `kategoris` dan `users`, **urutan pembuatan tabel menjadi penting**. Pastikan file migration kategoris memiliki timestamp (nama file) yang lebih awal dibandingkan file migration dokumens, agar tabel `kategoris` terbentuk lebih dulu sebelum tabel `dokumens` mencoba mereferensikannya. Secara default Artisan sudah mengurutkan berdasarkan waktu pembuatan file, sehingga selama Kategori dibuat sebelum Dokumen, urutan ini otomatis benar. Jika Anda membuat migration dalam urutan terbalik, MySQL akan error dengan pesan foreign key constraint fails.

3.5 Menjalankan Migration

Setelah kedua struktur tabel selesai didefinisikan, jalankan perintah berikut untuk benar-benar membuat tabelnya di dalam database `db_siarsip`:

```
php artisan migrate
```

Jika proses berhasil, Laravel akan menampilkan daftar migration yang sudah dijalankan, dan tabel `kategoris` beserta `dokumens` akan langsung tampil di phpMyAdmin lengkap dengan relasi foreign key-nya.

Beberapa perintah migration lain yang berguna selama pengembangan:

Perintah	Fungsi
`php artisan migrate:status`	Melihat migration mana yang sudah dijalankan dan mana yang masih pending.
`php artisan migrate:rollback`	Membatalkan batch migration terakhir (menjalankan method `down()`).
`php artisan migrate:refresh`	Me-rollback seluruh migration lalu menjalankannya ulang — berguna saat bereksperimen dengan struktur tabel.
`php artisan migrate:fresh`	Menghapus semua tabel lalu migrate ulang dari nol. Hati-hati: seluruh data hilang!

> ⚠ **CATATAN PENTING**
>
> `migrate:fresh` dan `migrate:refresh` menghapus seluruh data. Aman dipakai saat development dengan data dummy, tetapi **jangan pernah** dijalankan di server production yang berisi arsip sungguhan.

3.6 Mendefinisikan Relasi dan Mass Assignment pada Model

Migration mengurus struktur tabel; Model mengurus cara kita berinteraksi dengan data di dalamnya. Agar Model dapat digunakan untuk operasi CRUD dan relasi antar tabel, kita perlu mendefinisikan properti `$fillable` (untuk mass assignment) serta method relasi Eloquent pada masing-masing Model.

Apa itu Mass Assignment?

Mass assignment adalah teknik mengisi banyak kolom sekaligus dari sebuah array, misalnya `Dokumen::create($request->all())`. Ini praktis, tetapi berbahaya jika tidak dikontrol: pengguna nakal bisa menyisipkan field tambahan (misalnya `user_id` milik orang lain) melalui form. Laravel mengatasinya dengan properti `$fillable` — daftar putih kolom yang **boleh** diisi secara massal. Kolom di luar daftar ini akan diabaikan secara otomatis.

a. Model Kategori

▸ app/Models/Kategori.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Kategori extends Model
{
    /**
     * Kolom yang boleh diisi secara mass assignment.
     *
     * @var array
     */
    protected $fillable = [
        'nama_kategori',
        'deskripsi',
    ];

    /**
     * Relasi: satu kategori memiliki banyak dokumen.
     */
    public function dokumens(): HasMany
    {
        return $this->hasMany(Dokumen::class);
    }
}
```

b. Model Dokumen

▸ app/Models/Dokumen.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Dokumen extends Model
{
    /**
     * Kolom yang boleh diisi secara mass assignment.
     *
     * @var array
     */
    protected $fillable = [
        'kategori_id',
        'user_id',
        'nomor_arsip',
        'judul_dokumen',
        'deskripsi',
        'nama_file',
        'tipe_file',
        'ukuran_file',
        'tanggal_arsip',
    ];

    /**
     * Casting tipe data otomatis.
     *
     * @var array
     */
    protected $casts = [
        'tanggal_arsip' => 'date',
    ];

    /**
     * Relasi: dokumen dimiliki oleh satu kategori.
     */
    public function kategori(): BelongsTo
    {
        return $this->belongsTo(Kategori::class);
    }

    /**
     * Relasi: dokumen diunggah oleh satu user.
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

Bagian Kode	Penjelasan
`$fillable`	Daftar putih kolom yang boleh diisi melalui mass assignment, misalnya `Kategori::create($data)`. Kolom `id`, `created_at`, dan `updated_at` tidak perlu dimasukkan karena diisi otomatis oleh Laravel. Perhatikan `nomor_arsip` dan `nama_file` ikut didaftarkan karena keduanya dibuat oleh sistem lalu disimpan via `create()` pada Bab 5.
`$casts`	Memastikan kolom `tanggal_arsip` otomatis dikonversi menjadi objek Carbon (tanggal) setiap kali diambil dari database, sehingga dapat langsung diformat pada view, misalnya `$dokumen->tanggal_arsip->format('d-m-Y')`. Tanpa cast, nilainya hanya string biasa.
`hasMany(Dokumen::class)`	Relasi satu-ke-banyak dari sisi Kategori — satu kategori bisa memiliki banyak dokumen. Laravel otomatis menebak foreign key-nya adalah `kategori_id` di tabel `dokumens`. Contoh pakai: `$kategori->dokumens` mengembalikan seluruh dokumen dalam kategori tersebut.
`belongsTo(Kategori::class)`	Kebalikannya dari sisi Dokumen — satu dokumen dimiliki oleh satu kategori. Contoh pakai: `$dokumen->kategori->nama_kategori` untuk menampilkan nama kategori sebuah dokumen.
`belongsTo(User::class)`	Relasi ke Model User bawaan Laravel. Contoh pakai: `$dokumen->user->name` untuk mengetahui siapa yang mengarsipkan dokumen tersebut — menjawab kebutuhan audit SIARSIP.

💡 CARA MENGHAFAL HASMANY VS BELONGSTO

Tanyakan: "di tabel mana foreign key-nya berada?" Model yang **memiliki** foreign key memakai `belongsTo` (Dokumen punya `kategori_id`, maka Dokumen belongsTo Kategori). Model di sisi satunya memakai `hasMany`. Aturan sederhana ini berlaku untuk semua relasi one-to-many.

📝 Rangkuman Bab 3

- **Model** = representasi tabel sebagai class PHP via Eloquent ORM; **Migration** = version control untuk skema database.
- `php artisan make:model Kategori -m` membuat Model + Migration sekaligus; konvensi nama Model tunggal → tabel jamak.
- Tabel `dokumens` memakai `foreignId()->constrained()` untuk foreign key, dengan `restrict` untuk kategori dan `cascade` untuk user.
- Urutan migration mengikuti timestamp nama file — tabel induk (kategoris) harus dibuat sebelum tabel anak (dokumens).
- `$fillable` melindungi dari mass assignment yang tidak diinginkan; `$casts` mengonversi tipe data otomatis.
- Relasi: Kategori `hasMany` Dokumen; Dokumen `belongsTo` Kategori dan User.

✏ LATIHAN BAB 3

1. Buat migration untuk tabel `kategoris` dan `dokumens` sesuai bab ini, lalu jalankan `php artisan migrate`. Verifikasi di phpMyAdmin bahwa foreign key terbentuk.
2. Melalui `php artisan tinker`, coba buat satu kategori lalu tampilkan dengan `Kategori::first()`.
3. Apa yang terjadi jika Anda menghapus `'nama_kategori'` dari `$fillable` lalu mencoba `Kategori::create(['nama_kategori' => 'Tes'])`? Jelaskan mengapa.

BAB 4

Menampilkan Data dari Database

🎯 **TUJUAN PEMBELAJARAN**

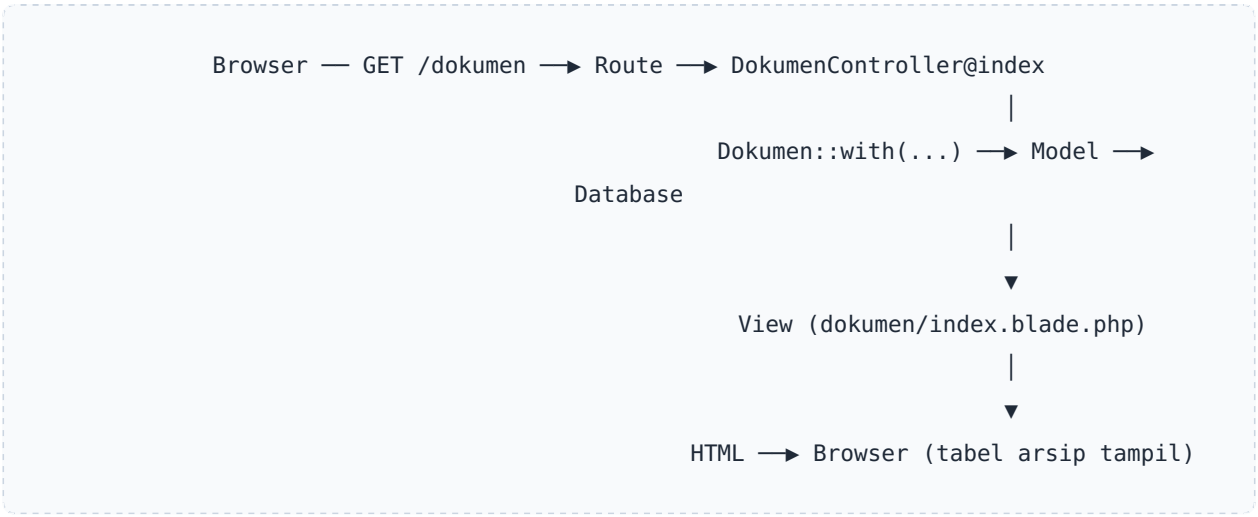
- Menjelaskan alur MVC (Model-View-Controller) pada Laravel.
- Membuat Controller dengan Artisan, termasuk opsi `--resource`.
- Mendaftarkan route resource dan route tambahan yang dilindungi middleware `auth`.
- Membuat layout Blade utama yang dipakai seluruh halaman SIARSIP.
- Menampilkan daftar kategori dan daftar dokumen lengkap dengan pencarian, filter, dan pagination.

4.1 Alur MVC yang Digunakan

Laravel menganut pola arsitektur **MVC** (Model-View-Controller) — pemisahan tanggung jawab menjadi tiga lapisan agar kode tetap terorganisir meski aplikasi membesar:

- **Model** — berurusan dengan data. Mengambil, menyimpan, dan mendefinisikan relasi. (Sudah kita buat pada Bab 3.)
- **View** — berurusan dengan tampilan. Template Blade yang menghasilkan HTML untuk dilihat pengguna.
- **Controller** — menjadi perantara. Menerima permintaan, meminta data ke Model, lalu meneruskannya ke View yang tepat.

Untuk menampilkan data arsip, alur yang dilalui adalah: **Route** menerima permintaan dari browser → meneruskannya ke **Controller** → Controller mengambil data melalui **Model/Eloquent** → data dikirim ke **View** untuk ditampilkan sebagai HTML.



Pola inilah yang akan kita terapkan pada fitur menampilkan daftar kategori dan daftar dokumen arsip. Setiap kali Anda bingung "kode ini seharusnya ditulis di mana?", kembalikan ke diagram ini: logika data di Model/Controller, HTML di View, pemetaan URL di Route.

4.2 Membuat Controller

Buat dua Controller menggunakan Artisan, masing-masing untuk mengelola Kategori dan Dokumen:

```
php artisan make:controller KategoriController --resource
php artisan make:controller DokumenController --resource
```

Opsi `--resource` membuat Controller yang sudah berisi kerangka tujuh method standar CRUD (`index, create, store, show, edit, update, destroy`), sehingga kita tinggal mengisi logikanya. Tujuh method ini adalah konvensi RESTful yang dipakai di seluruh ekosistem Laravel — menghafalnya akan sangat membantu membaca kode orang lain.

Method	Tugas	Contoh pada SIARSIP
`index()`	Menampilkan daftar data	Daftar seluruh arsip dokumen
`create()`	Menampilkan form tambah data	Form "Tambah Arsip Dokumen"
`store()`	Menyimpan data baru	Memproses form + upload berkas

`show()`	Menampilkan satu data detail	Halaman detail satu dokumen
`edit()`	Menampilkan form ubah data	Form "Edit Arsip Dokumen"
`update()`	Menyimpan perubahan data	Memproses form edit
`destroy()`	Menghapus data	Menghapus arsip + berkas fisiknya

4.3 Mendaftarkan Route

Route adalah "peta" yang menghubungkan URL dengan kode yang menanganinya. Buka file `routes/web.php`, kemudian daftarkan resource route untuk Kategori dan Dokumen, ditambah satu route khusus untuk proses unduh berkas:

▸ routes/web.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\KategoriController;
use App\Http\Controllers\DokumenController;

Route::get('/', function () {
    return redirect()->route('dokumen.index');
});

Route::middleware(['auth'])->group(function () {
    Route::resource('kategori', KategoriController::class);
    Route::resource('dokumen', DokumenController::class);
    Route::get('/dokumen/{dokumen}/download', [DokumenController::class, 'download'])
        ->name('dokumen.download');
});

require __DIR__.'/auth.php';
```

Baris Kode	Penjelasan
`Route::resource('kategori', ...)`	Secara otomatis mendaftarkan **tujuh route CRUD sekaligus** (index, create, store, show, edit, update, destroy) untuk resource kategori, sesuai konvensi RESTful Laravel. Satu baris menggantikan tujuh baris route manual — dan nama route-nya otomatis mengikuti pola `kategori.index`, `kategori.create`, dst.

`Route::middleware(['auth'])->group(...)`	Membungkus seluruh route arsip dengan middleware `auth`, sehingga **hanya pengguna yang sudah login** dapat mengakses modul arsip dokumen. Ini sangat penting karena setiap dokumen tercatat memiliki `user_id` pengunggah — tanpa login, sistem tidak tahu siapa yang mengarsipkan.
`Route::get('/dokumen/{dokumen}/download', ...)`	Route tambahan di luar resource standar, khusus untuk menangani proses unduh berkas dokumen (dibahas pada Bab 6). Parameter `{dokumen}` akan otomatis diubah menjadi object Model Dokumen via route model binding.
`Route::get('/', ...)`	Mengarahkan halaman utama (`/`) langsung ke daftar dokumen — halaman yang paling sering dibuka petugas arsip.

> ⚠ **CATATAN PENTING**
>
> Modul ini mengasumsikan sistem autentikasi (login/register) sudah tersedia — misalnya melalui starter kit **Laravel Breeze** — sehingga tabel `users` dan proses login sudah berjalan. Pembahasan instalasi starter kit autentikasi berada di luar cakupan modul ini karena fokus kita adalah konsep Model, Migration, dan CRUD. Baris `require __DIR__.'/auth.php';` adalah file route bawaan Breeze untuk halaman login/register.

> 💡 **TIPS**
>
> Untuk melihat seluruh daftar route yang terdaftar beserta method dan namanya, jalankan `php artisan route:list`. Perintah ini sangat membantu saat debugging "kok route-nya tidak ketemu?".

4.4 Membuat Layout Utama (Master Layout)

Agar tampilan konsisten di seluruh halaman — navbar yang sama, pesan sukses yang sama — kita buat satu **layout utama** menggunakan template Blade. Setiap halaman cukup "mengisi" bagian kontennya saja via `@section`, tanpa mengulang navbar di setiap file.

▸ resources/views/layouts/app.blade.php

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>SIARSIP - @yield('title', 'Sistem Arsip Dokumen')</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
    <nav class="navbar navbar-dark bg-dark px-3">
        <span class="navbar-brand mb-0 h1">SIARSIP</span>
        <div>
            <a href="{{ route('dokumen.index') }}" class="btn btn-sm btn-outline-light
me-2">Dokumen</a>
            <a href="{{ route('kategori.index') }}" class="btn btn-sm btn-outline-
light">Kategori</a>
        </div>
    </nav>
    <div class="container py-4">
        @if (session('success'))
            <div class="alert alert-success">{{ session('success') }}</div>
        @endif
        @if (session('error'))
            <div class="alert alert-danger">{{ session('error') }}</div>
        @endif
        @yield('content')
    </div>
</body>
</html>
```

Perhatikan tiga hal penting pada layout ini:

- `@yield('content')` adalah "slot kosong" yang akan diisi oleh masing-masing halaman. Inilah inti konsep layout: kerangka ditulis sekali, isi berbeda-beda per halaman.
- Blok `@if (session('success'))` menampilkan **flash message** — pesan sekali tampil setelah redirect (misalnya "Dokumen berhasil diarsipkan"). Karena diletakkan di layout, semua halaman otomatis bisa menampilkan pesan tanpa kode tambahan.
- Penggunaan `route('dokumen.index')` alih-alih URL mentah (`/dokumen`) membuat link tidak rusak meskipun URL route diubah di masa depan — praktik yang rapi dan tahan lama.

4.5 Menampilkan Daftar Kategori

Isi method `index()` pada KategoriController untuk mengambil seluruh data kategori beserta jumlah dokumen pada masing-masing kategori:

▸ app/Http/Controllers/KategoriController.php

```
public function index()
{
    $kategoris = Kategori::withCount('dokumens')->latest()->paginate(10);
    return view('kategori.index', compact('kategoris'));
}
```

- `withCount('dokumens')` memanfaatkan relasi `hasMany` dari Bab 3 untuk menghitung jumlah dokumen di setiap kategori **dalam satu query**, tanpa perlu query tambahan secara manual. Hasilnya bisa diakses melalui `$kategori->dokumens_count`. Bagi petugas arsip, angka ini langsung menjawab "kategori mana yang paling banyak dokumennya?"
- `latest()` mengurutkan dari data terbaru berdasarkan `created_at` — kategori yang baru dibuat muncul paling atas.
- `paginate(10)` membagi hasil menjadi halaman-halaman berisi 10 baris. Tanpa pagination, daftar ribuan kategori akan membuat halaman lambat dan sulit dinavigasi.

▸ resources/views/kategori/index.blade.php

```
@extends('layouts.app')

@section('content')
<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Data Kategori Arsip</h4>
    <a href="{{ route('kategori.create') }}" class="btn btn-primary">+ Tambah Kategori</
a>
</div>
<table class="table table-bordered bg-white">
    <thead>
        <tr>
            <th>No</th>
            <th>Nama Kategori</th>
            <th>Deskripsi</th>
            <th>Jumlah Dokumen</th>
            <th width="160">Aksi</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($kategoris as $kategori)
        <tr>
            <td>{{ $loop->iteration }}</td>
            <td>{{ $kategori->nama_kategori }}</td>
            <td>{{ $kategori->deskripsi ?? '-' }}</td>
            <td>{{ $kategori->dokumens_count }}</td>
            <td>
                <a href="{{ route('kategori.edit', $kategori->id) }}" class="btn btn-sm
btn-warning">Edit</a>
                <form action="{{ route('kategori.destroy', $kategori->id) }}"
method="POST" class="d-inline"
                      onsubmit="return confirm('Yakin ingin menghapus kategori ini?')">
                    @csrf
                    @method('DELETE')
                    <button type="submit" class="btn btn-sm btn-danger">Hapus</button>
                </form>
            </td>
        </tr>
        @empty
        <tr><td colspan="5" class="text-center">Belum ada data kategori.</td></tr>
        @endforelse
    </tbody>
</table>
{{ $kategoris->links() }}
@endsection
```

Beberapa sintaks Blade yang perlu dipahami:

- `@forelse ... @empty ... @endforelse` adalah perulangan yang memiliki "rencana cadangan": jika datanya kosong, baris `@empty` yang tampil ("Belum ada data kategori.") alih-alih tabel kosong yang membingungkan.

- `{{ $kategori->deskripsi ?? '-' }}` memakai operator ?? (null coalescing): jika deskripsi kosong (null), tampilkan tanda strip. Kecil, tetapi membuat tampilan jauh lebih rapi.
- `{{ $kategoris->links() }}` me-render tombol navigasi halaman (1, 2, 3, ...) secara otomatis dari hasil `paginate()`.
- Form hapus memakai `@csrf` (token keamanan anti-CSRF) dan `@method('DELETE')` — akan dibahas mendalam pada Bab 8.

4.6 Menampilkan Daftar Dokumen Arsip

Halaman utama SIARSIP adalah daftar dokumen. Selain menampilkan data, halaman ini membutuhkan dua fitur penting bagi petugas arsip: **pencarian** (berdasarkan judul/nomor arsip) dan **filter kategori**. Keduanya diimplementasikan di Controller agar View tetap bersih:

▸ app/Http/Controllers/DokumenController.php

```
public function index(Request $request)
{
    $dokumens = Dokumen::with(['kategori', 'user'])
        ->when($request->cari, function ($query, $cari) {
            $query->where('judul_dokumen', 'like', "%{$cari}%")
                  ->orWhere('nomor_arsip', 'like', "%{$cari}%");
        })
        ->when($request->kategori_id, function ($query, $kategoriId) {
            $query->where('kategori_id', $kategoriId);
        })
        ->latest('tanggal_arsip')
        ->paginate(10)
        ->withQueryString();

    $kategoris = Kategori::orderBy('nama_kategori')->get();

    return view('dokumen.index', compact('dokumens', 'kategoris'));
}
```

Bagian Kode	Penjelasan
`with(['kategori', 'user'])`	**Eager loading** — mengambil data kategori dan user terkait dalam query tambahan yang efisien. Tanpa ini, setiap baris tabel akan memicu query baru saat `$dokumen->kategori` dipanggil (masalah N+1 query: 1 query daftar +

	N query relasi untuk N baris). Dengan eager loading, 10 baris hanya butuh 3 query total, bukan 21.
`when($request->cari, ...)`	Menambahkan kondisi pencarian **hanya apabila** parameter `cari` dikirim via form/URL. Controller tetap ringkas tanpa if-else manual. Pencarian mencakup judul dokumen atau nomor arsip — dua hal yang paling diingat petugas saat mencari arsip.
`when($request->kategori_id, ...)`	Filter berdasarkan kategori yang dipilih pada dropdown. Kombinasi pencarian + filter dapat dipakai bersamaan.
`latest('tanggal_arsip')`	Mengurutkan berdasarkan tanggal arsip terbaru — dokumen yang baru diarsipkan muncul paling atas, sesuai kebiasaan kerja petugas.
`withQueryString()`	Memastikan parameter pencarian dan filter **tetap terbawa** saat berpindah halaman pagination. Tanpa ini, klik halaman 2 akan menghilangkan hasil pencarian — bug yang membingungkan pengguna.

▸ `resources/views/dokumen/index.blade.php`

```
@extends('layouts.app')

@section('content')
<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Data Arsip Dokumen</h4>
    <a href="{{ route('dokumen.create') }}" class="btn btn-primary">+ Tambah Arsip</a>
</div>
<form method="GET" class="row g-2 mb-3">
    <div class="col-md-5">
        <input type="text" name="cari" value="{{ request('cari') }}" class="form-
control"
               placeholder="Cari judul atau nomor arsip...">
    </div>
    <div class="col-md-4">
        <select name="kategori_id" class="form-select">
            <option value="">Semua Kategori</option>
            @foreach ($kategoris as $kategori)
            <option value="{{ $kategori->id }}" @selected(request('kategori_id') ==
$kategori->id)>
                {{ $kategori->nama_kategori }}
            </option>
            @endforeach
        </select>
    </div>
    <div class="col-md-3">
        <button type="submit" class="btn btn-secondary w-100">Filter</button>
    </div>
</form>
<table class="table table-bordered bg-white align-middle">
    <thead>
        <tr>
            <th>Nomor Arsip</th>
            <th>Judul Dokumen</th>
            <th>Kategori</th>
            <th>Tanggal Arsip</th>
            <th>Diunggah Oleh</th>
            <th width="180">Aksi</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($dokumens as $dokumen)
        <tr>
            <td><span class="badge bg-secondary">{{ $dokumen->nomor_arsip }}</span></td>
            <td>{{ $dokumen->judul_dokumen }}</td>
            <td>{{ $dokumen->kategori->nama_kategori }}</td>
            <td>{{ $dokumen->tanggal_arsip->format('d-m-Y') }}</td>
            <td>{{ $dokumen->user->name }}</td>
            <td>
                <a href="{{ route('dokumen.show', $dokumen->id) }}" class="btn btn-sm
btn-info">Detail</a>
                <a href="{{ route('dokumen.edit', $dokumen->id) }}" class="btn btn-sm
btn-warning">Edit</a>
            </td>
        </tr>
```

```
        @empty
        <tr><td colspan="6" class="text-center">Belum ada data arsip dokumen.</td></tr>
        @endforelse
    </tbody>
</table>
{{ $dokumens->links() }}
@endsection
```

Perhatikan detail-detail kecil yang membuat halaman ini "terasa" profesional:

- Form pencarian memakai `method="GET"` (bukan POST) karena pencarian tidak mengubah data — dan URL hasil pencarian bisa di-bookmark atau dibagikan ke rekan kerja.
- `value="{{ request('cari') }}"` dan `@selected(...)` mempertahankan isian form setelah halaman di-refresh — pengguna tidak perlu mengetik ulang kata kunci setiap kali.
- Nomor arsip ditampilkan sebagai badge agar mudah dipindai mata saat petugas mencari nomor tertentu di antara puluhan baris.
- `$dokumen->tanggal_arsip->format('d-m-Y')` bekerja berkat `$casts` pada Model (Bab 3) — contoh nyata mengapa cast itu penting.

📝 Rangkuman Bab 4

- Alur MVC: Route → Controller → Model → View. Setiap lapisan punya tanggung jawab yang jelas.
- `make:controller --resource` menghasilkan kerangka 7 method CRUD standar.
- `Route::resource()` mendaftarkan 7 route sekaligus; middleware `auth` melindungi seluruh area arsip.
- Layout Blade (`@extends` / `@yield`) menjaga konsistensi tampilan dan menampung flash message terpusat.
- `withCount`, `with` (eager loading), `when`, dan `withQueryString` adalah senjata utama menampilkan data yang efisien dan ramah pengguna.

✏ LATIHAN BAB 4

1. Jalankan `php artisan route:list` dan identifikasi 7 route yang dihasilkan oleh `Route::resource('dokumen', ...)` beserta method HTTP-nya.
2. Tambahkan kolom “Tipe Berkas” pada tabel daftar dokumen yang menampilkan `$dokumen->tipe_file` dalam huruf kapital.
3. Uji fitur pencarian: cari dengan kata kunci yang ada, lalu kata kunci yang tidak ada. Perhatikan pesan `@empty` yang tampil.

BAB 5

Insert Data (Tambah Arsip Dokumen)

🎯 TUJUAN PEMBELAJARAN

- Menjelaskan alur form → validasi → penyimpanan pada Laravel.
- Membuat fitur tambah kategori dengan validasi dan pesan error.
- Membuat fitur tambah dokumen arsip: validasi file, nomor arsip otomatis, dan penyimpanan berkas dengan nama unik (UUID).
- Memahami peran `enctype="multipart/form-data"`, `old()`, dan `@error`.

Menambah data adalah operasi **Create** dalam CRUD. Di Laravel, alurnya selalu sama dan perlu dihafal di luar kepala:

```
1. GET /dokumen/create ⟶ Controller@create ⟶ tampilkan form kosong
        2. User mengisi form, klik Simpan ⟶ POST /dokumen
3. Controller@store ⟶ VALIDASI input ⟶ gagal? kembali ke form + pesan error
                                              ⟶ lolos? simpan ke DB ⟶
                          redirect + pesan sukses
```

Tahap validasi adalah "penjaga gerbang": tidak ada data yang boleh masuk ke database sebelum lolos aturan. Untuk SIARSIP, ini krusial — arsip yang datanya cacat (judul kosong, file tidak didukung) akan menjadi masalah bertahun-tahun kemudian.

5.1 Tambah Data Kategori

Sebelum menambahkan dokumen, kategori dokumen perlu tersedia terlebih dahulu karena kolom `kategori_id` bersifat wajib (foreign key) — tidak mungkin mengarsipkan surat ke kategori yang belum ada. Lengkapi method `create()` dan `store()` pada KategoriController:

▸ app/Http/Controllers/KategoriController.php

```
public function create()
{
    return view('kategori.create');
}

public function store(Request $request)
{
    $request->validate([
        'nama_kategori' => 'required|string|max:100|unique:kategoris,nama_kategori',
        'deskripsi'     => 'nullable|string',
    ]);

    Kategori::create($request->only(['nama_kategori', 'deskripsi']));

    return redirect()->route('kategori.index')
        ->with('success', 'Kategori berhasil ditambahkan.');
}
```

Aturan Validasi	Penjelasan & Kaitan dengan SIARSIP
`required`	Nama kategori wajib diisi — kategori tanpa nama tidak ada gunanya dan merusak dropdown filter.
`string|max:100`	Harus berupa teks dan maksimal 100 karakter, selaras dengan definisi kolom `VARCHAR(100)` di migration. Validasi dan skema database harus selalu "sepakat".
`unique:kategoris,nama_kategori`	Nama kategori tidak boleh duplikat. Tanpa ini, bisa muncul dua kategori "Surat Masuk" yang membingungkan filter dan laporan.
`nullable`	Deskripsi boleh kosong — selaras dengan `->nullable()` di migration.

Perhatikan `$request->only(['nama_kategori', 'deskripsi'])`: kita hanya mengambil field yang dibutuhkan, bukan seluruh input. Ini adalah lapisan pertahanan tambahan selain `$fillable` — prinsip defense in depth.

▸ resources/views/kategori/create.blade.php

```
@extends('layouts.app')

@section('content')
<h4 class="mb-3">Tambah Kategori</h4>
<form action="{{ route('kategori.store') }}" method="POST" class="bg-white p-4 rounded
shadow-sm">
    @csrf
    <div class="mb-3">
        <label class="form-label">Nama Kategori</label>
        <input type="text" name="nama_kategori" value="{{ old('nama_kategori') }}"
               class="form-control @error('nama_kategori') is-invalid @enderror">
        @error('nama_kategori') <div class="invalid-feedback">{{ $message }}</div>
@enderror
    </div>
    <div class="mb-3">
        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi') }}
</textarea>
    </div>
    <button type="submit" class="btn btn-primary">Simpan</button>
    <a href="{{ route('kategori.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection
```

- `@csrf` menghasilkan token keamanan tersembunyi di form. Laravel menolak request POST tanpa token valid — melindungi SIARSIP dari serangan Cross-Site Request Forgery di mana situs jahat mencoba mengirim form atas nama pengguna yang sedang login.
- `old('nama_kategori')` mengisi kembali input dengan nilai sebelumnya jika validasi gagal — pengguna tidak perlu mengetik ulang dari nol. Detail kecil, pengalaman besar.
- `@error('nama_kategori') ... @enderror` menampilkan pesan error tepat di bawah field yang bermasalah, sekaligus menambahkan class `is-invalid` agar border input memerah.

5.2 Tambah Data Dokumen Arsip (dengan Upload File)

Ini adalah bagian inti dari studi kasus Sistem Arsip Dokumen, karena setiap dokumen wajib disertai berkas fisik yang diunggah. Ada tiga hal khusus yang perlu ditangani

pada proses ini — ketiganya adalah keputusan desain yang membedakan "aplikasi arsip" dari "CRUD biasa":

- **Nomor arsip** (`nomor_arsip`) dibuat otomatis oleh sistem, bukan diinput manual oleh pengguna, sehingga formatnya konsisten dan tidak mudah tertukar. Format: `ARS-YYYYMMDD-0001`.
- **Berkas yang diunggah disimpan dengan nama unik** menggunakan `Str::uuid()`, sedangkan judul dokumen (`judul_dokumen`) tetap menyimpan nama/perihal yang mudah dibaca manusia. Ini mencegah dua file dengan nama asli yang sama saling menimpa di server.
- **Kolom `tipe_file` dan `ukuran_file` diisi otomatis** berdasarkan metadata file yang diunggah, tidak diinput manual — menghilangkan risiko salah catat.

Lengkapi method `create()` pada DokumenController untuk menampilkan form, lengkap dengan daftar kategori sebagai pilihan dropdown:

▸ app/Http/Controllers/DokumenController.php

```
public function create()
{
    $kategoris = Kategori::orderBy('nama_kategori')->get();
    return view('dokumen.create', compact('kategoris'));
}
```

Selanjutnya lengkapi method `store()` dengan logika validasi, pembuatan nomor arsip otomatis, dan penyimpanan berkas ke disk `public`:

▸ app/Http/Controllers/DokumenController.php

```
use Illuminate\Support\Str;
use Illuminate\Support\Facades\Storage;

public function store(Request $request)
{
    $request->validate([
        'kategori_id'    => 'required|exists:kategoris,id',
        'judul_dokumen'  => 'required|string|max:255',
        'deskripsi'      => 'nullable|string',
        'tanggal_arsip'  => 'required|date',
        'file'           => 'required|file|mimes:pdf,doc,docx,xls,xlsx,jpg,jpeg,png|max:
10240',
    ]);

    // 1. Buat nomor arsip otomatis, format: ARS-YYYYMMDD-0001
    $tanggal = now()->format('Ymd');
    $urutan  = Dokumen::whereDate('created_at', now())->count() + 1;
    $nomorArsip = 'ARS-' . $tanggal . '-' . str_pad($urutan, 4, '0', STR_PAD_LEFT);

    // 2. Simpan berkas dengan nama unik ke storage/app/public/dokumen
    $file = $request->file('file');
    $ekstensi = $file->getClientOriginalExtension();
    $namaFileTersimpan = Str::uuid() . '.' . $ekstensi;
    $file->storeAs('dokumen', $namaFileTersimpan, 'public');

    // 3. Simpan data ke database
    Dokumen::create([
        'kategori_id'   => $request->kategori_id,
        'user_id'       => auth()->id(),
        'nomor_arsip'   => $nomorArsip,
        'judul_dokumen' => $request->judul_dokumen,
        'deskripsi'     => $request->deskripsi,
        'nama_file'     => $namaFileTersimpan,
        'tipe_file'     => $ekstensi,
        'ukuran_file'   => $file->getSize(),
        'tanggal_arsip' => $request->tanggal_arsip,
    ]);

    return redirect()->route('dokumen.index')
        ->with('success', 'Dokumen berhasil diarsipkan dengan nomor ' . $nomorArsip);
}
```

Bagian Kode	Penjelasan
`exists:kategoris,id`	Memastikan `kategori_id` yang dikirim benar-benar ada di tabel kategoris — mencegah ID palsu hasil manipulasi form.
`mimes:...|max:10240`	Membatasi jenis berkas yang boleh diunggah hanya format dokumen dan gambar umum, dengan ukuran

	maksimal 10240 KB (10 MB), untuk mencegah penyalahgunaan upload (misalnya mengunggah file executable).
`Dokumen::whereDate('created_at', now())->count() + 1`	Menghitung jumlah dokumen yang sudah diarsipkan pada hari yang sama, lalu menambah 1 sebagai nomor urut berikutnya, sehingga nomor arsip selalu unik per hari. Dikombinasikan dengan constraint `unique` di database sebagai pengaman ganda.
`str_pad($urutan, 4, '0', STR_PAD_LEFT)`	Mengubah angka urutan menjadi 4 digit dengan tambahan angka nol di depan, misalnya 1 menjadi 0001, agar format nomor arsip rapi dan konsisten — penting saat nomor arsip dicetak pada label fisik map arsip.
`Str::uuid() . '.' . $ekstensi`	Membuat nama file unik menggunakan UUID (Universally Unique Identifier) sehingga dua berkas dengan nama asli sama (misalnya dua-duanya bernama "surat.pdf") tidak akan saling menimpa satu sama lain di server.
`$file->storeAs('dokumen', $namaFileTersimpan, 'public')`	Menyimpan berkas ke folder `storage/app/public/dokumen`, yang dapat diakses publik melalui `public/storage/dokumen` berkat symbolic link dari Bab 2. Parameter ketiga (`'public'`) memilih disk penyimpanan.
`auth()->id()`	Mengambil ID pengguna yang sedang login untuk disimpan sebagai `user_id` — inilah "jejak audit" siapa yang mengarsipkan dokumen tersebut. Middleware `auth` (Bab 4) menjamin nilainya tidak pernah null.

▸ `resources/views/dokumen/create.blade.php`

```
@extends('layouts.app')

@section('content')
<h4 class="mb-3">Tambah Arsip Dokumen</h4>
<form action="{{ route('dokumen.store') }}" method="POST" enctype="multipart/form-data"
      class="bg-white p-4 rounded shadow-sm">
    @csrf
    <div class="mb-3">
        <label class="form-label">Kategori</label>
        <select name="kategori_id" class="form-select @error('kategori_id') is-invalid
@enderror">
            <option value="">-- Pilih Kategori --</option>
            @foreach ($kategoris as $kategori)
            <option value="{{ $kategori->id }}" @selected(old('kategori_id') ==
$kategori->id)>
                {{ $kategori->nama_kategori }}
            </option>
            @endforeach
        </select>
        @error('kategori_id') <div class="invalid-feedback">{{ $message }}</div>
@enderror
    </div>
    <div class="mb-3">
        <label class="form-label">Judul Dokumen</label>
        <input type="text" name="judul_dokumen" value="{{ old('judul_dokumen') }}"
               class="form-control @error('judul_dokumen') is-invalid @enderror">
        @error('judul_dokumen') <div class="invalid-feedback">{{ $message }}</div>
@enderror
    </div>
    <div class="mb-3">
        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi') }}
</textarea>
    </div>
    <div class="mb-3">
        <label class="form-label">Tanggal Arsip</label>
        <input type="date" name="tanggal_arsip" value="{{ old('tanggal_arsip') }}"
               class="form-control @error('tanggal_arsip') is-invalid @enderror">
        @error('tanggal_arsip') <div class="invalid-feedback">{{ $message }}</div>
@enderror
    </div>
    <div class="mb-3">
        <label class="form-label">Berkas Dokumen (PDF/Word/Excel/Gambar, maks 10MB)</
label>
        <input type="file" name="file" class="form-control @error('file') is-invalid
@enderror">
        @error('file') <div class="invalid-feedback">{{ $message }}</div> @enderror
    </div>
    <button type="submit" class="btn btn-primary">Simpan Arsip</button>
    <a href="{{ route('dokumen.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection
```

⚠ CATATAN PENTING

Perhatikan atribut `enctype="multipart/form-data"` pada tag `<form>`. Atribut ini **wajib** ditambahkan setiap kali form memiliki input bertipe file. Tanpa atribut ini, browser mengirim form sebagai teks biasa sehingga data berkas tidak ikut terkirim, dan `$request->file('file')` akan selalu bernilai `null` — validasi `required` pun akan selalu gagal dengan pesan yang membingungkan.

📝 Rangkuman Bab 5

- Alur Create: form (GET) → validasi (POST) → simpan → redirect + flash message.
- `@csrf`, `old()`, dan `@error` adalah trio wajib form Laravel yang aman dan ramah pengguna.
- Tiga keputusan desain SIARSIP: nomor arsip otomatis (`ARS-YYYYMMDD-0001`), nama file unik via UUID, dan metadata file (tipe & ukuran) yang diisi otomatis.
- Validasi file memakai `mimes` + `max`; form upload wajib memakai `enctype="multipart/form-data"`.
- `auth()->id()` mencatat siapa pengunggah — fondasi audit trail arsip.

✏ LATIHAN BAB 5

1. Tambahkan 3 kategori (Surat Masuk, Surat Keluar, Sertifikat), lalu arsipkan 2 dokumen dengan berkas berbeda. Verifikasi di phpMyAdmin bahwa `nomor_arsip`, `nama_file`, `tipe_file`, dan `ukuran_file` terisi otomatis.
2. Coba unggah file berekstensi `.exe` (atau ubah nama file teks menjadi `.exe`). Apa yang terjadi? Jelaskan aturan validasi mana yang menolaknya.
3. Sengaja hapus atribut `enctype` dari form, lalu coba simpan. Catat perilaku yang terjadi dan jelaskan penyebabnya.

BAB 6

Menampilkan Detail Data dan Unduh Berkas

🎯 **TUJUAN PEMBELAJARAN**

- Menampilkan halaman detail satu dokumen arsip beserta metadata berkasnya.
- Memahami `findOrFail()` dan route model binding.
- Membuat fitur unduh berkas yang aman: verifikasi keberadaan file dan nama unduhan yang rapi.
- Menampilkan preview gambar langsung di halaman detail.

6.1 Menampilkan Detail Dokumen

Halaman daftar (Bab 4) hanya menampilkan ringkasan. Untuk kebutuhan kerja nyata — misalnya pimpinan ingin memastikan isi arsip sebelum mengunduhnya — setiap baris data perlu dapat dibuka detailnya secara lengkap, termasuk metadata berkas (tipe file dan ukuran file) serta tombol unduh.

Lengkapi method `show()` pada DokumenController:

▸ app/Http/Controllers/DokumenController.php

```
public function show(string $id)
{
    $dokumen = Dokumen::with(['kategori', 'user'])->findOrFail($id);
    return view('dokumen.show', compact('dokumen'));
}
```

- `findOrFail($id)` mencari dokumen berdasarkan primary key. Jika tidak ditemukan — misalnya pengguna mengetik manual URL `/dokumen/99999` — Laravel otomatis menampilkan halaman error **404** yang rapi, tanpa kita perlu menulis pengecekan `if` manual.
- `with(['kategori', 'user'])` kembali memakai eager loading agar nama kategori dan nama pengunggah tidak memicu query tambahan.

- resources/views/dokumen/show.blade.php

```
@extends('layouts.app')

@section('content')
<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Detail Arsip Dokumen</h4>
    <a href="{{ route('dokumen.index') }}" class="btn btn-secondary btn-sm">&larr;
Kembali</a>
</div>
<div class="card">
    <div class="card-body">
        <table class="table table-borderless mb-0">
            <tr>
                <th width="220">Nomor Arsip</th>
                <td>: <span class="badge bg-secondary">{{ $dokumen->nomor_arsip }}</
span></td>
            </tr>
            <tr>
                <th>Judul Dokumen</th>
                <td>: {{ $dokumen->judul_dokumen }}</td>
            </tr>
            <tr>
                <th>Kategori</th>
                <td>: {{ $dokumen->kategori->nama_kategori }}</td>
            </tr>
            <tr>
                <th>Deskripsi</th>
                <td>: {{ $dokumen->deskripsi ?? '-' }}</td>
            </tr>
            <tr>
                <th>Tanggal Arsip</th>
                <td>: {{ $dokumen->tanggal_arsip->format('d F Y') }}</td>
            </tr>
            <tr>
                <th>Diunggah Oleh</th>
                <td>: {{ $dokumen->user->name }}</td>
            </tr>
            <tr>
                <th>Tipe Berkas</th>
                <td>: {{ strtoupper($dokumen->tipe_file) }}</td>
            </tr>
            <tr>
                <th>Ukuran Berkas</th>
                <td>: {{ number_format($dokumen->ukuran_file / 1024, 2) }} KB</td>
            </tr>
        </table>
        @if (in_array($dokumen->tipe_file, ['jpg', 'jpeg', 'png']))
        <div class="my-3">
            <img src="{{ asset('storage/dokumen/' . $dokumen->nama_file) }}"
                 class="img-fluid rounded border" style="max-height:320px" alt="Preview
Dokumen">
        </div>
        @endif
        <a href="{{ route('dokumen.download', $dokumen->id) }}" class="btn btn-primary">
            Unduh Berkas
```

```
        </a>
    </div>
</div>
@endsection
```

Detail-detail yang perlu diperhatikan:

- `$dokumen->tanggal_arsip->format('d F Y')` menampilkan tanggal dalam format panjang yang mudah dibaca manusia (misalnya "05 Agustus 2026"), berbeda dengan format pendek `d-m-Y` pada halaman daftar. Fleksibilitas ini dimungkinkan oleh `$casts` pada Model.
- `number_format($dokumen->ukuran_file / 1024, 2)` mengonversi ukuran dari byte ke kilobyte dengan 2 desimal — karena "2.450,75 KB" jauh lebih bermakna bagi pengguna daripada "2509056 byte".
- Blok `@if (in_array(...))` menampilkan **preview gambar** langsung di halaman detail, tetapi hanya untuk tipe gambar. Untuk PDF/Word, preview tidak ditampilkan — pengguna memakai tombol unduh. Ini contoh progressive enhancement: fitur tambahan yang muncul hanya jika relevan.
- Fungsi `asset()` menghasilkan URL lengkap menuju file di folder public — di sini menunjuk ke `public/storage/dokumen/...` yang merupakan symbolic link dari Bab 2.

> **⚠ CATATAN PENTING**
>
> Kode `{{ asset('storage/dokumen/' . $dokumen->nama_file) }}` hanya akan berfungsi apabila perintah `php artisan storage:link` pada Bab 2 telah dijalankan, karena folder `public/storage` merupakan symbolic link menuju `storage/app/public`. Jika preview gambar rusak (ikon gambar patah), inilah hal pertama yang perlu diperiksa.

6.2 Mengunduh Berkas Dokumen

Untuk mengunduh berkas, kita punya dua pilihan desain:

1. **Link langsung** ke `/storage/dokumen/namafile.pdf` — sederhana, tetapi nama file UUID yang acak terekspos ke pengguna, dan tidak ada kesempatan untuk memeriksa apapun sebelum file disajikan.

2. **Melalui Controller** — satu route khusus yang memeriksa keberadaan file, lalu menyajikannya dengan nama unduhan yang rapi.

Modul ini memilih pendekatan kedua karena lebih profesional dan lebih mudah dikembangkan (misalnya kelak ditambah pengecekan hak akses per dokumen). Tambahkan method berikut pada DokumenController — route-nya sudah kita daftarkan pada Bab 4:

▸ app/Http/Controllers/DokumenController.php

```
public function download(Dokumen $dokumen)
{
    $path = 'dokumen/' . $dokumen->nama_file;

    if (! Storage::disk('public')->exists($path)) {
        abort(404, 'Berkas tidak ditemukan di server.');
    }

    $namaUnduhan = Str::slug($dokumen->judul_dokumen) . '.' . $dokumen->tipe_file;

    return Storage::disk('public')->download($path, $namaUnduhan);
}
```

Bagian Kode	Penjelasan
`download(Dokumen $dokumen)`	Menggunakan **Route Model Binding** — Laravel otomatis mengambil data Dokumen berdasarkan `{dokumen}` pada URL tanpa perlu `findOrFail()` manual. Jika ID tidak ada, otomatis 404. Perhatikan bedanya dengan `show(string $id)`: di sini parameternya langsung berupa object Model.
`Storage::disk('public')->exists($path)`	Memastikan berkas benar-benar masih ada secara fisik sebelum diunduh. Ini mengantisipasi kondisi data vs file tidak sinkron: misalnya berkas terhapus manual dari server, tetapi datanya masih tercatat di database. Tanpa pengecekan ini, pengguna akan mendapat error mentah yang membingungkan.
`Str::slug($dokumen->judul_dokumen)`	Mengubah judul dokumen menjadi format nama file yang aman (huruf kecil, spasi menjadi strip, simbol khusus dibuang), misalnya "Surat Undangan Rapat!" menjadi `surat-undangan-rapat.pdf`. Saat diunduh, pengguna melihat nama file yang deskriptif, bukan

	kode UUID acak — detail yang membedakan aplikasi asal jadi dengan aplikasi yang dipikirkan matang.
`Storage::disk('public')->download(...)`	Method bawaan Laravel untuk mengirimkan file sebagai response unduhan (force download) lengkap dengan header `Content-Disposition` yang tepat, sehingga browser menampilkan dialog "Save As" alih-alih mencoba membuka file di tab.

💡 KAPAN MEMAKAI ROUTE MODEL BINDING?

Route model binding cocok ketika parameter URL adalah ID dari sebuah Model dan Anda butuh object-nya langsung. Ia menghemat satu baris `findOrFail` dan membuat signature method lebih ekspresif: `download(Dokumen $dokumen)` langsung terbaca sebagai "unduh dokumen ini". Syaratnya: nama parameter route (`{dokumen}`) harus sama dengan nama variabel (`$dokumen`).

📝 Rangkuman Bab 6

- Halaman detail memakai `findOrFail()` (otomatis 404 jika data tidak ada) dan eager loading relasi.
- Metadata berkas disajikan ramah manusia: tanggal panjang, ukuran dalam KB, tipe dalam huruf kapital, plus preview gambar kondisional.
- Fitur unduh dirancang melalui Controller (bukan link langsung) dengan **route model binding**, verifikasi keberadaan file, dan nama unduhan yang rapi via `Str::slug()`.

✏ LATIHAN BAB 6

1. Buka halaman detail sebuah dokumen, lalu ubah ID pada URL menjadi angka yang tidak ada. Perhatikan halaman 404 yang tampil.
2. Hapus manual sebuah file di `storage/app/public/dokumen` (tanpa menghapus datanya), lalu klik tombol unduh. Pesan apa yang muncul? Jelaskan baris kode mana yang menghasilkannya.
3. Unduh sebuah dokumen dan perhatikan nama file yang tersimpan di komputer Anda. Bandingkan dengan nama file yang tersimpan di server — jelaskan perbedaannya dan alasannya.

BAB 7

Edit dan Update Data

🎯 TUJUAN PEMBELAJARAN

- Menjelaskan alur edit → update dan perbedaan aturan validasinya dengan proses insert.
- Membuat fitur edit kategori, termasuk pengecualian `unique` untuk data yang sedang diedit.
- Membuat fitur edit dokumen dengan berkas yang bersifat **opsional** dan pembersihan file lama.
- Memahami `@method('PUT')` dan `old('field', $model->field)`.

Operasi **Update** sekilas mirip dengan Create: ada form, ada validasi, ada penyimpanan. Namun ada dua perbedaan mendasar yang membuatnya layak dibahas dalam bab tersendiri:

1. **Form harus terisi data lama.** Pengguna tidak mengisi dari nol, melainkan mengubah sebagian field. Setiap input harus menampilkan nilai saat ini via `old('field', $model->field)`.
2. **Aturan validasi perlu penyesuaian.** Contoh: aturan `unique` harus mengecualikan baris yang sedang diedit, dan file upload menjadi opsional (pengguna tidak wajib mengunggah ulang berkas).

7.1 Edit dan Update Data Kategori

Proses edit kategori relatif sederhana karena tidak melibatkan berkas. Lengkapi method `edit()` dan `update()` pada KategoriController:

▸ app/Http/Controllers/KategoriController.php

```
public function edit(string $id)
{
    $kategori = Kategori::findOrFail($id);
    return view('kategori.edit', compact('kategori'));
}

public function update(Request $request, string $id)
{
    $kategori = Kategori::findOrFail($id);

    $request->validate([
        'nama_kategori' => 'required|string|max:100|unique:kategoris,nama_kategori,' .
$kategori->id,
        'deskripsi'     => 'nullable|string',
    ]);

    $kategori->update($request->only(['nama_kategori', 'deskripsi']));

    return redirect()->route('kategori.index')
        ->with('success', 'Kategori berhasil diperbarui.');
}
```

> **⚠ CATATAN PENTING**
>
> Perhatikan aturan `unique:kategoris,nama_kategori,' . $kategori->id` — parameter ketiga ini memberitahu Laravel untuk **mengecualikan baris kategori yang sedang diedit** dari pengecekan unique. Tanpa pengecualian ini, pengguna yang membuka form edit lalu menekan "Update" tanpa mengubah apapun akan mendapat error "nama kategori sudah digunakan" — oleh datanya sendiri! Ini adalah bug klasik yang wajib dihindari.

Perhatikan juga `$kategori->update(...)` berbeda dengan `Kategori::create(...)` pada Bab 5: `update()` dipanggil pada **instance** yang sudah diambil dari database, sehingga Laravel tahu baris mana yang harus diubah (berdasarkan primary key-nya).

▸ resources/views/kategori/edit.blade.php

```
@extends('layouts.app')

@section('content')
<h4 class="mb-3">Edit Kategori</h4>
<form action="{{ route('kategori.update', $kategori->id) }}" method="POST"
      class="bg-white p-4 rounded shadow-sm">
    @csrf
    @method('PUT')
    <div class="mb-3">
        <label class="form-label">Nama Kategori</label>
        <input type="text" name="nama_kategori" value="{{ old('nama_kategori',
$kategori->nama_kategori) }}"
               class="form-control @error('nama_kategori') is-invalid @enderror">
        @error('nama_kategori') <div class="invalid-feedback">{{ $message }}</div>
@enderror
    </div>
    <div class="mb-3">
        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi',
$kategori->deskripsi) }}</textarea>
    </div>
    <button type="submit" class="btn btn-primary">Update</button>
    <a href="{{ route('kategori.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection
```

- `@method('PUT')` — HTML form secara native hanya mendukung GET dan POST. Laravel menyediakan directive ini untuk HTTP method spoofing: request dikirim sebagai POST tetapi dikenali router sebagai PUT, sesuai route `kategori.update` dari resource controller. Tanpa ini, request akan ditolak dengan error 405 (Method Not Allowed).
- `old('nama_kategori', $kategori->nama_kategori)` — pola dua lapis: jika validasi gagal dan form di-refresh, tampilkan input terakhir pengguna (`old()`); jika form baru dibuka, tampilkan data dari database. Inilah cara yang benar mengisi form edit.

7.2 Edit dan Update Data Dokumen (Berkas Bersifat Opsional)

Pada proses edit dokumen, terdapat aturan khusus yang mencerminkan realitas kerja petugas arsip: **pengguna tidak wajib mengunggah ulang berkas**. Seringkali yang perlu diperbaiki hanya judul yang salah ketik atau kategori yang keliru — memaksa upload ulang file 10 MB hanya untuk memperbaiki satu huruf adalah desain yang buruk.

Aturannya:

- Apabila **tidak ada** berkas baru yang dipilih → pertahankan berkas lama (kolom `nama_file`, `tipe_file`, `ukuran_file` tidak diubah).
- Apabila **ada** berkas baru → hapus berkas lama dari storage (agar tidak menumpuk file yatim), simpan berkas baru dengan nama UUID baru, dan perbarui ketiga kolom metadata.

▸ app/Http/Controllers/DokumenController.php

```
public function edit(string $id)
{
    $dokumen = Dokumen::findOrFail($id);
    $kategoris = Kategori::orderBy('nama_kategori')->get();
    return view('dokumen.edit', compact('dokumen', 'kategoris'));
}

public function update(Request $request, string $id)
{
    $dokumen = Dokumen::findOrFail($id);

    $request->validate([
        'kategori_id'    => 'required|exists:kategoris,id',
        'judul_dokumen'  => 'required|string|max:255',
        'deskripsi'      => 'nullable|string',
        'tanggal_arsip'  => 'required|date',
        'file'           => 'nullable|file|mimes:pdf,doc,docx,xls,xlsx,jpg,jpeg,png|max:
10240',
    ]);

    $data = $request->only(['kategori_id', 'judul_dokumen', 'deskripsi',
'tanggal_arsip']);

    // Jika pengguna mengunggah berkas baru, ganti berkas lama
    if ($request->hasFile('file')) {
        if (Storage::disk('public')->exists('dokumen/' . $dokumen->nama_file)) {
            Storage::disk('public')->delete('dokumen/' . $dokumen->nama_file);
        }

        $file = $request->file('file');
        $ekstensi = $file->getClientOriginalExtension();
        $namaFileTersimpan = Str::uuid() . '.' . $ekstensi;
        $file->storeAs('dokumen', $namaFileTersimpan, 'public');

        $data['nama_file']   = $namaFileTersimpan;
        $data['tipe_file']   = $ekstensi;
        $data['ukuran_file'] = $file->getSize();
    }

    $dokumen->update($data);

    return redirect()->route('dokumen.index')
        ->with('success', 'Dokumen berhasil diperbarui.');
}
```

Bagian Kode	Penjelasan
`'file' => 'nullable|file|...'`	Berbeda dengan aturan pada `store()` yang bersifat `required`, di sini memakai `nullable` karena file boleh tidak

	dikirim ulang oleh pengguna. Satu kata ini mengubah seluruh perilaku form.
`$request->only([...])` tanpa file	Array `$data` awal hanya berisi kolom teks. Kolom file sengaja **tidak** dimasukkan dulu — prinsipnya: jangan sentuh apa yang tidak berubah.
`if ($request->hasFile('file'))`	Blok ini hanya dijalankan apabila pengguna benar-benar memilih berkas baru. Jika tidak, `$data` tidak memiliki key `nama_file`/`tipe_file`/`ukuran_file`, sehingga kolom tersebut tidak ikut ter-update dan nilai lama tetap dipertahankan.
`Storage::disk('public')->delete(...)`	Menghapus berkas lama dari server **sebelum** menyimpan berkas baru. Urutan ini penting: jika penyimpanan file baru gagal, file lama sudah terlanjur hilang — namun untuk modul ini, kesederhanaan didahulukan; pada aplikasi production, pertimbangkan membungkusnya dalam database transaction.

▸ `resources/views/dokumen/edit.blade.php`

```
@extends('layouts.app')

@section('content')
<h4 class="mb-3">Edit Arsip Dokumen</h4>
<form action="{{ route('dokumen.update', $dokumen->id) }}" method="POST"
enctype="multipart/form-data"
      class="bg-white p-4 rounded shadow-sm">
    @csrf
    @method('PUT')
    <div class="mb-3">
        <label class="form-label">Kategori</label>
        <select name="kategori_id" class="form-select">
            @foreach ($kategoris as $kategori)
            <option value="{{ $kategori->id }}"
                @selected(old('kategori_id', $dokumen->kategori_id) == $kategori->id)>
                {{ $kategori->nama_kategori }}
            </option>
            @endforeach
        </select>
    </div>
    <div class="mb-3">
        <label class="form-label">Judul Dokumen</label>
        <input type="text" name="judul_dokumen"
               value="{{ old('judul_dokumen', $dokumen->judul_dokumen) }}" class="form-
control">
    </div>
    <div class="mb-3">
        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi',
$dokumen->deskripsi) }}</textarea>
    </div>
    <div class="mb-3">
        <label class="form-label">Tanggal Arsip</label>
        <input type="date" name="tanggal_arsip"
               value="{{ old('tanggal_arsip', $dokumen->tanggal_arsip->format('Y-m-
d')) }}" class="form-control">
    </div>
    <div class="mb-3">
        <label class="form-label">Berkas Saat Ini</label>
        <p class="mb-1">
            <a href="{{ route('dokumen.download', $dokumen->id) }}" target="_blank">
                {{ $dokumen->judul_dokumen }}.{{ $dokumen->tipe_file }}
            </a>
        </p>
        <label class="form-label">Ganti Berkas (kosongkan jika tidak ingin mengubah)</
label>
        <input type="file" name="file" class="form-control">
    </div>
    <button type="submit" class="btn btn-primary">Update</button>
    <a href="{{ route('dokumen.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection
```

Tiga detail penting pada form edit dokumen:

- **Menampilkan berkas saat ini** dengan link unduh — pengguna perlu tahu file apa yang sedang terpasang sebelum memutuskan menggantinya. Tanpa ini, form edit terasa seperti "menebak-nebak".
- Label "Ganti Berkas (kosongkan jika tidak ingin mengubah)" mengomunikasikan dengan eksplisit bahwa upload bersifat opsional — jangan biarkan pengguna menebak-nebak perilaku form.
- `$dokumen->tanggal_arsip->format('Y-m-d')` — input HTML `type="date"` hanya menerima format `YYYY-MM-DD`. Format lain (misalnya `d-m-Y`) akan membuat input tampil kosong meskipun datanya ada. Sekali lagi, `$casts` pada Model yang memungkinkan pemanggilan `->format()` ini.

📝 Rangkuman Bab 7

- Form edit memakai pola `old('field', $model->field)` dan `@method('PUT')` untuk method spoofing.
- Aturan `unique` pada update harus mengecualikan baris yang sedang diedit.
- Pada dokumen, file bersifat opsional (`nullable`): blok penggantian file hanya jalan jika `hasFile('file')`, dan file lama dihapus agar tidak menjadi yatim.
- Form edit yang baik menampilkan data saat ini (termasuk link berkas lama) dan menjelaskan dengan eksplisit field mana yang opsional.

✏ LATIHAN BAB 7

1. Edit sebuah dokumen: ubah hanya judulnya tanpa mengganti berkas. Verifikasi di folder `storage/app/public/dokumen` bahwa file lama masih ada dan `nama_file` di database tidak berubah.
2. Edit dokumen yang sama lagi, kali ini dengan mengganti berkas. Verifikasi bahwa file lama terhapus dan file baru tersimpan dengan nama UUID baru.
3. Coba buka form edit kategori, langsung tekan "Update" tanpa mengubah apapun. Seharusnya tidak ada error validasi — jelaskan mengapa (petunjuk: parameter ketiga aturan `unique`).

BAB 8

Delete Data (Hapus Arsip)

🎯 TUJUAN PEMBELAJARAN

- Menjelaskan mengapa operasi hapus membutuhkan penanganan khusus pada data berelasi.
- Menghapus kategori dengan menangani constraint `restrict` secara elegan (pesan ramah, bukan error 500).
- Menghapus dokumen beserta berkas fisiknya agar tidak menyisakan file yatim.
- Menambahkan konfirmasi hapus dan memahami `@method('DELETE')`.

Menghapus data terlihat seperti operasi paling sederhana dalam CRUD — nyatanya, justru di sinilah keputusan desain database (Bab 1 dan 3) diuji. Dua skenario berbeda membutuhkan dua penanganan berbeda:

- **Hapus kategori** → terhalang `onDelete('restrict')` jika masih ada dokumen terkait. Kita harus menangani penolakan database dengan pesan yang ramah.
- **Hapus dokumen** → tidak terhalang constraint apapun, tetapi berkas fisik di storage harus ikut dibersihkan manual — database tidak tahu-menahu soal file.

8.1 Menghapus Data Kategori (Menangani Relasi Restrict)

Ingat kembali keputusan desain pada Bab 1: kolom `kategori_id` memakai `onDelete('restrict')`. Artinya, ketika seseorang mencoba menghapus kategori "Surat Masuk" yang masih menaungi 50 dokumen, **MySQL akan menolak** dan melemparkan `Illuminate\Database\QueryException`.

Jika exception ini tidak ditangani, pengguna akan melihat halaman error 500 yang menakutkan — padahal dari sudut pandang bisnis, ini bukan error, melainkan aturan yang bekerja sebagaimana mestinya: "kategori ini tidak boleh dihapus karena masih dipakai". Tugas kita adalah menerjemahkan penolakan teknis tersebut menjadi pesan yang dimengerti petugas arsip:

▸ app/Http/Controllers/KategoriController.php

```
use Illuminate\Database\QueryException;

public function destroy(string $id)
{
    $kategori = Kategori::findOrFail($id);

    try {
        $kategori->delete();
    } catch (QueryException $e) {
        return redirect()->route('kategori.index')
            ->with('error', 'Kategori "' . $kategori->nama_kategori . '" tidak dapat
dihapus '
                . 'karena masih memiliki dokumen arsip yang terkait.');
    }

    return redirect()->route('kategori.index')
        ->with('success', 'Kategori berhasil dihapus.');
}
```

- Blok `try-catch` menangkap `QueryException` yang dilempar database saat constraint `restrict` dilanggar, lalu mengarahkan kembali ke daftar kategori dengan flash message `error` — yang otomatis tampil sebagai alert merah berkat layout pada Bab 4.
- Pesan error menyebut **nama kategorinya** dan **alasannya** ("masih memiliki dokumen arsip yang terkait"), plus implikasi solusinya: pindahkan atau hapus dulu dokumen-dokumen tersebut.

> **⚠ CATATAN PENTING**
>
> Sebagai alternatif yang lebih eksplisit dan tidak bergantung pada exception database, kategori juga bisa dicek terlebih dahulu menggunakan `if ($kategori->dokumens()->exists())` sebelum `delete()` dijalankan. Pendekatan try-catch pada contoh di atas dipilih agar aturan integritas data tetap **dijamin langsung oleh database** (constraint), sehingga lebih aman meskipun ada jalur kode lain yang mencoba menghapus data secara langsung — misalnya via tinker atau seeder.

> 💡 **PELAJARAN DESAIN**
>
> Inilah buah dari keputusan `restrict` vs `cascade` pada Bab 1. Jika kategori memakai `cascade`, menghapus satu kategori akan melenyapkan seluruh dokumen di dalamnya tanpa peringatan — bencana untuk arsip. `restrict` memaksa penghapusan dilakukan secara sadar: bereskan dulu dokumennya, baru hapus kategorinya.

8.2 Menghapus Data Dokumen Beserta Berkas Fisiknya

Berbeda dengan kategori, penghapusan dokumen tidak dibatasi oleh foreign key manapun (tidak ada tabel lain yang mereferensikan `dokumens`). Namun ada pekerjaan rumah yang tidak kalah penting: **berkas fisik di storage harus ikut terhapus**. Database hanya menyimpan nama file, bukan file-nya — jika kita hanya menghapus baris database, file akan tertinggal selamanya di server sebagai orphan file yang diam-diam menghabiskan ruang penyimpanan.

▸ app/Http/Controllers/DokumenController.php

```
public function destroy(string $id)
{
    $dokumen = Dokumen::findOrFail($id);

    if (Storage::disk('public')->exists('dokumen/' . $dokumen->nama_file)) {
        Storage::disk('public')->delete('dokumen/' . $dokumen->nama_file);
    }

    $dokumen->delete();

    return redirect()->route('dokumen.index')
        ->with('success', 'Arsip dokumen berhasil dihapus.');
}
```

- Pengecekan `exists()` sebelum `delete()` membuat kode tahan terhadap kondisi file yang sudah hilang duluan (misalnya terhapus manual) — penghapusan data tetap berjalan tanpa error.
- **Urutan penting:** hapus file dulu, baru hapus baris database. Jika dibalik dan penghapusan file gagal di tengah jalan, kita kehilangan jejak nama file-nya (karena baris database sudah hilang) dan file menjadi yatim permanen.

8.3 Menambahkan Konfirmasi Hapus pada View

Penghapusan bersifat permanen — tidak ada "recycle bin" di SIARSIP. Untuk mencegah penghapusan yang tidak disengaja (misalnya salah klik), setiap tombol hapus dibungkus form dengan **konfirmasi JavaScript** sederhana. Tambahkan potongan berikut pada kolom Aksi di `resources/views/dokumen/index.blade.php`:

▸ resources/views/dokumen/index.blade.php (tambahan kolom aksi)

```
<form action="{{ route('dokumen.destroy', $dokumen->id) }}" method="POST" class="d-
inline"
      onsubmit="return confirm('Yakin ingin menghapus arsip \'' + '{{ $dokumen-
>judul_dokumen }}' + '\'? Berkas terkait juga akan terhapus permanen.')">
    @csrf
    @method('DELETE')
    <button type="submit" class="btn btn-sm btn-danger">Hapus</button>
</form>
```

- Fungsi `confirm()` menampilkan dialog "OK / Cancel" bawaan browser. Jika pengguna menekan Cancel, `onsubmit` mengembalikan `false` dan form **tidak dikirim**.
- Pesan konfirmasi menyebut **judul dokumennya** dan menegaskan **"berkas terkait juga akan terhapus permanen"** — pengguna membuat keputusan dengan informasi lengkap, bukan sekadar "Yakin ingin menghapus?" yang generik.
- `@method('DELETE')` diperlukan karena HTML form secara native hanya mendukung method GET dan POST. Laravel menyediakan Blade directive ini untuk HTTP method spoofing, sehingga permintaan tetap dikirim sebagai POST namun dikenali oleh Router Laravel sebagai method DELETE sesuai dengan resource route yang telah didaftarkan pada Bab 4.
- `@csrf` tetap wajib — tanpa token ini, request hapus akan ditolak dengan error 419 (Page Expired).

> 💡 **UNTUK PENGEMBANGAN LANJUTAN**
>
> Pada sistem arsip production, pertimbangkan **soft delete** (`$table->softDeletes()` + trait `SoftDeletes` pada Model) sebagai pengganti hapus permanen. Data yang "dihapus" hanya ditandai `deleted_at` sehingga masih bisa dipulihkan — fitur yang sangat masuk akal untuk arsip yang bersifat historis.

📝 Rangkuman Bab 8

- Hapus kategori memakai try-catch `QueryException` untuk menangani penolakan `restrict` dengan pesan yang ramah pengguna.
- Hapus dokumen wajib membersihkan berkas fisik di storage (cek `exists()` dulu), dengan urutan: file dulu, baru baris database.
- Tombol hapus memakai form POST + `@method('DELETE')` + `@csrf` + dialog konfirmasi yang informatif.
- Keputusan `restrict` vs `cascade` pada Bab 1 terbukti nilainya di bab ini: integritas arsip terjaga.

✏ LATIHAN BAB 8

1. Coba hapus kategori yang masih memiliki dokumen. Pesan apa yang tampil? Lalu pindahkan/hapus dokumennya dan coba hapus kategorinya lagi.
2. Hapus sebuah dokumen, lalu periksa folder `storage/app/public/dokumen` — pastikan file fisiknya ikut hilang.
3. Tekan tombol Hapus lalu pilih "Cancel" pada dialog konfirmasi. Jelaskan mekanisme apa yang membatalkan pengiriman form.

BAB 9

Penyempurnaan: Validasi, Keamanan, dan Akses

🎯 TUJUAN PEMBELAJARAN

- Merelokasi aturan validasi ke class Form Request agar Controller tetap ramping.
- Memahami peran middleware `auth` dalam melindungi data arsip.
- Menerapkan checklist keamanan upload berkas pada SIARSIP.
- Memahami mekanisme flash message terpusat.

Delapan bab sebelumnya membangun SIARSIP yang berfungsi. Bab ini membuatnya layak production: kode lebih rapi, akses lebih terkontrol, dan celah keamanan umum ditutup. Tiga topik ini adalah pembeda antara "aplikasi tugas kuliah" dan "aplikasi yang berani dipakai instansi".

9.1 Merapikan Validasi dengan Form Request

Perhatikan method `store()` dan `update()` pada DokumenController: keduanya memuat blok validasi yang panjang. Semakin banyak aturan, semakin gemuk Controller — padahal tugas Controller seharusnya fokus pada logika bisnis (membuat nomor arsip, menyimpan file), bukan pada daftar aturan validasi.

Solusi Laravel: pindahkan validasi ke class khusus bernama **Form Request**. Buat class-nya dengan Artisan:

```
php artisan make:request StoreDokumenRequest
```

▸ app/Http/Requests/StoreDokumenRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreDokumenRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check();
    }

    public function rules(): array
    {
        return [
            'kategori_id'   => 'required|exists:kategoris,id',
            'judul_dokumen' => 'required|string|max:255',
            'deskripsi'     => 'nullable|string',
            'tanggal_arsip' => 'required|date',
            'file'          => 'required|file|mimes:pdf,doc,docx,xls,xlsx,jpg,jpeg,png|
max:10240',
        ];
    }

    public function messages(): array
    {
        return [
            'file.mimes' => 'Berkas harus berformat PDF, Word, Excel, atau gambar (JPG/
PNG).',
            'file.max'   => 'Ukuran berkas tidak boleh lebih dari 10 MB.',
        ];
    }
}
```

Method	Penjelasan
`authorize()`	Menentukan **siapa yang boleh** memakai request ini. Mengembalikan `true` berarti semua pengguna yang login diizinkan. Jika mengembalikan `false`, Laravel otomatis menolak dengan error 403 (Forbidden) — bahkan sebelum validasi dijalankan. Inilah lapisan otorisasi paling awal.
`rules()`	Berisi seluruh aturan validasi — sama persis dengan yang sebelumnya ditulis di Controller. Memindahkannya ke sini membuat aturan dapat dipakai ulang (misalnya dipakai bersama oleh `store` dan `update` dengan sedikit modifikasi).

`messages()`	Pesan error kustom dalam Bahasa Indonesia. Pesan default Laravel berbahasa Inggris dan bernada teknis ("The file must be a file of type..."); pesan kustom membuat petugas arsip langsung paham apa yang salah tanpa perlu menerjemahkan.

Setelah class ini dibuat, method `store()` pada Controller cukup diganti signature-nya:

```
public function store(StoreDokumenRequest $request)  // sebelumnya: Request $request
{
    // $request->validate(...) TIDAK lagi diperlukan —
    // validasi sudah otomatis dijalankan Laravel SEBELUM method ini dieksekusi.
    ...
}
```

Dengan perubahan satu baris ini, Controller kehilangan ~10 baris kode validasi dan menjadi fokus pada tugas utamanya. Pola ini — thin controller, dedicated request class — adalah standar kode Laravel yang profesional.

> 💡 **KAPAN MEMBUAT FORM REQUEST?**
>
> Aturan praktisnya: jika blok validasi lebih dari 3–4 aturan, atau dipakai di lebih dari satu method (store + update), pindahkan ke Form Request. Untuk validasi satu-dua baris seperti pada KategoriController, validasi inline di Controller masih acceptable.

9.2 Middleware Autentikasi

Middleware adalah "penjaga pintu" yang memeriksa setiap request sebelum mencapai Controller. Analogi sederhananya: sebelum tamu bisa masuk ke ruang arsip, satpam memeriksa kartu identitasnya terlebih dahulu.

Seluruh route arsip dokumen wajib dilindungi middleware `auth`, karena kolom `user_id` pada tabel `dokumens` mensyaratkan adanya pengguna yang sedang login — tanpa login, sistem tidak bisa mencatat siapa pengunggahnya, dan data arsip bisa diubah siapa saja. Konfigurasi ini telah diterapkan pada Bab 4:

```
Route::middleware(['auth'])->group(function () {
    Route::resource('kategori', KategoriController::class);
    Route::resource('dokumen', DokumenController::class);
    // ...
});
```

Setiap request ke URL di dalam group akan melewati pemeriksaan: sudah login? Jika belum, pengguna otomatis diarahkan ke halaman login. Jika sudah, request diteruskan ke Controller.

Selain itu, pastikan setiap query data dokumen turut mempertimbangkan siapa yang login — terutama apabila di masa depan modul ini dikembangkan menjadi multi-user dengan hak akses berbeda-beda. Misalnya, petugas arsip biasa hanya boleh melihat dokumen yang ia unggah sendiri, sedangkan pimpinan boleh melihat semuanya. Kebutuhan semacam ini diselesaikan dengan lapisan otorisasi seperti **Laravel Policy** atau package **Spatie Laravel-Permission** (lihat Bab 10).

> 💡 **ANALOGI**
>
> Jika route adalah pintu-pintu ruangan di kantor arsip, maka middleware adalah satpam yang berjaga di setiap pintu. `auth` memeriksa "apakah Anda pegawai sini?", sedangkan policy/permission memeriksa "apakah Anda boleh masuk ke ruangan khusus ini?". Keduanya bekerja berlapis.

9.3 Catatan Keamanan Upload Berkas

Fitur upload adalah salah satu titik paling rawan dalam aplikasi web — ia membiarkan orang luar menaruh file ke server Anda. Untuk SIARSIP yang menerima berkas dari banyak pengguna, checklist berikut wajib dipatuhi:

- **Selalu validasi ekstensi dengan aturan `mimes`,** jangan hanya mengandalkan atribut `accept` pada input HTML atau nama field form — keduanya dapat dimanipulasi dengan mudah via developer tools. Validasi server-side adalah pertahanan yang sesungguhnya.
- **Jangan pernah menyimpan berkas dengan nama asli pengguna** (`getClientOriginalName()`) sebagai nama file di server. Selain berisiko tabrakan antar file, nama asli bisa mengandung karakter berbahaya atau path traversal

(misalnya `../../.env`). Gunakan nama unik seperti `Str::uuid()` sebagaimana diterapkan pada Bab 5.

- **Tentukan batas ukuran maksimal file** (`max:10240` = 10 MB pada modul ini) untuk menghindari penyalahgunaan penyimpanan server — tanpa batas, satu pengguna bisa mengunggah file raksasa dan melumpuhkan server.
- **Pilih disk penyimpanan sesuai sensitivitas dokumen.** Simpan berkas arsip di disk `public` hanya jika memang dokumen tersebut boleh diakses lewat tautan langsung. Untuk dokumen yang bersifat rahasia/terbatas, gunakan disk `local` (tidak bisa diakses via URL) dan sajikan berkas **melalui Controller** — seperti method `download()` pada Bab 6 — yang memverifikasi hak akses pengguna terlebih dahulu, bukan disk `public` yang dapat diakses siapa saja yang mengetahui URL-nya.
- **Pertimbangkan pemindaian antivirus** untuk file yang diunggah pada deployment production, terutama jika instansi menerima berkas dari pihak eksternal.

9.4 Flash Message

Sepanjang modul ini, setiap redirect setelah proses simpan, ubah, dan hapus data selalu disertai `->with('success', ...)` atau `->with('error', ...)`. Mekanismenya:

1. `->with('success', '...')` menyimpan pesan ke **session** yang hanya bertahan untuk **satu request berikutnya** — inilah arti "flash".
2. Setelah redirect, layout utama (`resources/views/layouts/app.blade.php`, Bab 4) memeriksa `session('success')` dan menampilkannya sebagai alert hijau; `session('error')` sebagai alert merah.
3. Pada request berikutnya (misalnya refresh halaman), pesan otomatis hilang — tidak menumpuk.

Karena logika tampilan pesan didefinisikan sekali di layout, Controller cukup fokus mengirim pesannya. Inilah contoh Don't Repeat Yourself (DRY): satu definisi, dipakai di seluruh halaman.

📝 Rangkuman Bab 9

- **Form Request** memindahkan validasi keluar dari Controller: `authorize()` untuk izin, `rules()` untuk aturan, `messages()` untuk pesan Bahasa Indonesia.
- **Middleware `auth`** menjaga seluruh area arsip hanya bisa diakses pengguna yang login; otorisasi lebih rinci memakai Policy/Permission.
- **Keamanan upload:** validasi `mimes` di server, nama file UUID (bukan nama asli), batas ukuran, dan pemilihan disk sesuai sensitivitas dokumen.
- **Flash message** memberi umpan balik sekali tampil setelah redirect, dirender terpusat di layout.

✏ LATIHAN BAB 9

1. Buat `StoreDokumenRequest` dan refactor method `store()` agar memakainya. Uji bahwa validasi tetap berjalan (coba simpan tanpa judul).
2. Logout dari aplikasi, lalu coba akses langsung URL `/dokumen`. Ke mana Anda diarahkan? Jelaskan middleware apa yang bekerja.
3. Tambahkan pesan kustom Bahasa Indonesia untuk aturan `judul_dokumen.required` dan `kategori_id.exists`, lalu uji.

BAB 10

Penutup

🎯 **TUJUAN PEMBELAJARAN**

- Mereview seluruh alur pembangunan SIARSIP dari awal hingga akhir.
- Menyelesaikan proyek mandiri sebagai uji pemahaman.
- Mengidentifikasi arah pengembangan lanjutan yang realistis.

10.1 Ringkasan

Melalui modul ini, kita telah membangun sebuah modul CRUD Sistem Informasi Arsip Dokumen (SIARSIP) secara lengkap menggunakan Laravel 13:

- **Bab 1** — Memahami konsep framework, keunggulan Laravel 13, skenario bisnis SIARSIP, dan rancangan database (tabel `kategoris`, `dokumens`, relasi ke `users`).
- **Bab 2** — Instalasi dan konfigurasi project: environment, `composer create-project`, `php artisan serve`, koneksi MySQL via `.env`, Bootstrap CDN, dan `php artisan storage:link`.
- **Bab 3** — Perancangan Model dan Migration untuk Kategori dan Dokumen, lengkap dengan foreign key, strategi `onDelete` restrict/cascade, `$fillable`, `$casts`, dan relasi `hasMany`/`belongsTo`.
- **Bab 4** — Menampilkan data dengan alur MVC: resource Controller, route yang dilindungi middleware `auth`, layout Blade, eager loading, pencarian, filter kategori, dan pagination.
- **Bab 5** — Insert data: validasi form, nomor arsip otomatis (`ARS-YYYYMMDD-0001`), upload berkas dengan nama UUID, dan pencatatan metadata file otomatis.
- **Bab 6** — Halaman detail dokumen, preview gambar, dan fitur unduh via Controller dengan route model binding dan nama file unduhan yang rapi.
- **Bab 7** — Edit dan update: pengecualian `unique` untuk data yang diedit, method spoofing PUT, dan penanganan berkas opsional beserta pembersihan file lama.
- **Bab 8** — Delete data: penanganan constraint `restrict` via try-catch, pembersihan berkas fisik, dan konfirmasi hapus yang informatif.

- **Bab 9** — Penyempurnaan: Form Request, middleware autentikasi, checklist keamanan upload, dan flash message terpusat.

Dengan menyelesaikan seluruh bab, Anda telah memahami alur kerja Laravel secara end-to-end pada kasus nyata yang melibatkan relasi antar tabel dan pengelolaan berkas — bukan sekadar CRUD teks biasa. Pola pikir yang sama (desain database → Model/Migration → Route/Controller/View → validasi → keamanan) dapat dipakai ulang untuk membangun aplikasi apapun: sistem inventaris, perpustakaan, kepegawaian, dan sebagainya.

10.2 Proyek Mandiri

Sebagai uji pemahaman, kembangkan SIARSIP Anda dengan fitur berikut tanpa panduan langkah demi langkah — gunakan bab-bab sebelumnya sebagai referensi:

> ✏ **PROYEK MANDIRI**
>
> 1. **Halaman dashboard sederhana** yang menampilkan: total dokumen, total kategori, dan 5 dokumen yang terakhir diarsipkan. (Petunjuk: Bab 4 — query agregat dan `latest()`.)
> 2. **Urutkan daftar dokumen** berdasarkan kolom yang bisa diklik (misalnya klik header "Tanggal Arsip" untuk toggle ascending/descending). (Petunjuk: Bab 4 — parameter query string.)
> 3. **Tambahkan kolom "status"** pada tabel dokumens (misalnya: aktif / dipinjam / dimusnahkan) lengkap dengan migration baru, form, dan filter di halaman daftar. (Petunjuk: Bab 3 — `make:migration` untuk menambah kolom; Bab 5 & 7 — form.)

10.3 Pengembangan Lanjutan

Sistem Arsip Dokumen yang telah dibangun dapat dikembangkan lebih lanjut, di antaranya:

- **Ekspor data arsip ke PDF atau Excel** untuk kebutuhan laporan bulanan/tahunan — misalnya rekapitulasi jumlah dokumen per kategori.

- **Pencatatan log aktivitas** (activity log) setiap kali dokumen ditambah, diubah, atau dihapus, agar histori pengelolaan arsip dapat ditelusuri — melengkapi kolom `user_id` yang sudah ada.
- **Penerapan role dan permission** (misalnya Admin, Petugas Arsip, dan Pimpinan) menggunakan package seperti Spatie Laravel-Permission, sehingga hak akses setiap fitur dapat diatur lebih rinci — misalnya hanya Admin yang boleh menghapus kategori.
- **Pencarian yang lebih canggih** (full text search) menggunakan Laravel Scout apabila jumlah data arsip sudah sangat besar dan pencarian `LIKE` mulai lambat.
- **Dukungan multi-file** dalam satu dokumen arsip, dengan membuat tabel relasi tambahan seperti `dokumen_lampirans` (satu dokumen bisa memiliki banyak lampiran).
- **Notifikasi email atau sistem** ketika ada dokumen baru yang diarsipkan pada kategori tertentu — misalnya memberi tahu pimpinan setiap ada kontrak baru.

10.4 Referensi

- Dokumentasi resmi Laravel 13 — https://laravel.com/docs (sumber utama dan terkini untuk seluruh fitur pada modul ini: installation, migrations, Eloquent, validation, filesystem, routing).
- Struktur alur belajar pada modul ini terinspirasi dari rangkaian Tutorial Laravel untuk Pemula, SantriKoding.com — https://santrikoding.com.

> ⚠ **CATATAN PENTING**
>
> Modul ini disusun ulang dan disesuaikan sepenuhnya (kode, struktur database, dan studi kasus) untuk kebutuhan pembuatan Sistem Arsip Dokumen, dan tidak menyalin isi tulisan dari sumber referensi di atas. Sumber referensi dicantumkan semata-mata sebagai penghormatan atas inspirasi alur belajar CRUD Laravel yang digunakan.

LAMPIRAN A

Glosarium Istilah

Istilah	Penjelasan
Artisan	Antarmuka baris perintah (CLI) resmi Laravel untuk membuat file, menjalankan migration, dan berbagai tugas otomatisasi.
Blade	Template engine Laravel untuk menulis tampilan HTML dengan sintaks ringkas seperti `@if`, `@foreach`, dan `{{ }}`.
CSRF	Cross-Site Request Forgery — serangan yang memalsukan request atas nama pengguna yang sedang login. Ditangkal dengan token `@csrf`.
Eager Loading	Teknik mengambil data relasi sekaligus di awal (via `with()`) untuk menghindari query berulang (masalah N+1).
Eloquent ORM	Pustaka Laravel yang memetakan tabel database menjadi class/ object PHP sehingga CRUD bisa dilakukan tanpa SQL mentah.
Foreign Key	Kolom yang merujuk ke primary key tabel lain; menjaga keterkaitan antar data (misalnya `kategori_id`).
Mass Assignment	Pengisian banyak kolom sekaligus dari array; dikontrol via properti `$fillable` demi keamanan.
Middleware	Lapisan pemeriksa request sebelum mencapai Controller, misalnya `auth` untuk memastikan pengguna sudah login.
Migration	File PHP berisi definisi perubahan skema database; berfungsi sebagai version control untuk database.
MVC	Model-View-Controller — pola arsitektur pemisahan logika data, tampilan, dan perantara.
N+1 Query	Masalah performa ketika 1 query daftar diikuti N query relasi; diatasi dengan eager loading.
Route Model Binding	Fitur Laravel yang otomatis mengubah parameter ID di URL menjadi object Model.

Seeder	Class untuk mengisi database dengan data awal/dummy secara otomatis.
Soft Delete	Teknik "menghapus" dengan menandai `deleted_at` alih-alih menghapus baris sungguhan, sehingga data bisa dipulihkan.
Symbolic Link	"Jalan pintas" level sistem file; dipakai `php artisan storage:link` agar folder storage bisa diakses via public.
UUID	Universally Unique Identifier — string acak yang praktis dijamin unik; dipakai sebagai nama file arsip.

LAMPIRAN B

Error Umum dan Solusinya

Sebelum panik saat menemukan error, cocokkan gejalanya dengan tabel berikut — sembilan dari sepuluh error pemula Laravel ada di sini:

Gejala / Pesan Error	Penyebab Umum & Solusi
`SQLSTATE[HY000] [2002] Connection refused`	MySQL belum dinyalakan. Nyalakan service MySQL di XAMPP/ Laragon, lalu ulangi perintah.
`SQLSTATE[HY000] [1049] Unknown database 'db_siarsip'`	Database belum dibuat. Buat database `db_siarsip` via phpMyAdmin terlebih dahulu.
`Access denied for user 'root'@'localhost'`	Username/password di `.env` salah. Sesuaikan `DB_USERNAME`/ `DB_PASSWORD`, lalu `php artisan config:clear`.
Perubahan `.env` tidak berpengaruh	Konfigurasi ter-cache. Jalankan `php artisan config:clear`.
`Foreign key constraint fails` saat migrate	Urutan migration salah: tabel anak dibuat sebelum tabel induk. Pastikan migration kategoris memiliki timestamp lebih awal dari dokumens.
`419 Page Expired` saat submit form	Token CSRF hilang/kedaluwarsa. Pastikan form memuat `@csrf`; jika sesi kedaluwarsa, refresh halaman.
`405 Method Not Allowed`	Method HTTP tidak cocok dengan route. Form edit/update/ hapus wajib memakai `@method('PUT')` / `@method('DELETE')`.
Upload selalu gagal / `$request->file('file')` null	Form lupa atribut `enctype="multipart/form-data"`.
Gambar preview rusak / unduh 404 padahal upload sukses	`php artisan storage:link` belum dijalankan.
`MassAssignmentException`	Kolom belum didaftarkan di `$fillable` pada Model.

Error `unique` saat update tanpa mengubah nama	Aturan `unique` belum mengecualikan ID baris yang sedang diedit (lihat Bab 7.1).
Halaman 500 setelah hapus kategori	`QueryException` dari constraint `restrict` tidak ditangani try-catch (lihat Bab 8.1).
`Target class [XController] does not exist`	Namespace/use statement salah atau file Controller belum dibuat. Periksa `use App\Http\Controllers\...;`.

Jika error tidak ada di tabel ini, baca pesan error dari **baris paling atas** — Laravel menampilkan jenis exception dan file penyebabnya di sana. Salin pesan tersebut ke mesin pencari beserta kata kunci "Laravel 13" untuk menemukan solusi komunitas.

— Selesai —

Terima kasih telah menyelesaikan modul ini.
Semoga SIARSIP yang Anda bangun bermanfaat bagi instansi Anda.